

РЕФЕРАТ

Курсовий проєкт на тему «Веб-платформа для вивчення мов програмування в ігровій формі» розроблено згідно з індивідуальним завданням, затвердженим на засіданні циклової комісії програмного забезпечення.

Основним завданням проєкту є створення веб-застосунку для вивчення мов програмування з інтерактивним функціоналом, системою тестування знань та відстеженням прогресу користувача.

Основні функції системи:

- реєстрація та авторизація користувачів;
- вибір мов програмування та навчальних курсів;
- проходження теоретичних матеріалів і тестових завдань;
- автоматична перевірка відповідей;
- система гейміфікації;
- адміністративна панель для керування контентом;
- розмежування прав доступу між адміністратором і користувачем.

Пояснювальна записка містить 53 сторінки, 35 рисунків.

Метою проєкту є створення зручної платформи, яка поєднує навчання та ігрові елементи для підвищення мотивації користувачів. Веб-застосунок дозволяє поєднати теоретичні матеріали, практичні завдання та систему винагород, що сприяє кращому засвоєнню інформації та контролю прогресу навчання.

Для реалізації веб-застосунку використано HTML, CSS та JavaScript для клієнтської частини, а також C# для серверної логіки. Розробка здійснювалась у середовищах PhpStorm та Visual Studio.

У результаті створено структуру веб-платформи для вивчення мов програмування, яка забезпечує взаємодію між користувачем, навчальними матеріалами та системою тестування, а також підтримує подальше розширення функціоналу.

3MICT

ВСТУП.....	3
1 ЗАГАЛЬНИЙ РОЗДІЛ.....	4
1.1 Постановка завдання.....	4
1.2 Опис вхідної інформації.....	4
1.3 Опис результуючої інформації.....	5
1.4 Стадії та етапи розробки задачі	6
1.5 Опис існуючих методів та рішень	8
2 РОЗРОБКА ТЕХНІЧНОГО ТА РОБОЧОГО ПРОЕКТУ	12
2.1 Опис та обґрунтування вибору структури та методу організації вихідних та вхідних даних.....	12
2.2 Розробка структурної схеми проєкту та об’єктної моделі задачі на основі UML діаграми	13
2.3 Розробка та опис програмної реалізації та тестування задачі	20
3 СПЕЦІАЛЬНИЙ РОЗДІЛ	29
3.1 Інструкція з інсталяції розробленого проєкту.....	29
3.2 Інструкція з експлуатації проєкту	33
Висновки	40
Список використаних джерел	41
ДОДАТКИ	43

					КП.ОК19.ПІ.232.44.000 ВП							
Змін.	Аркуш	№ докум.	Підпис	Дата								
Розробив	Онищук О. Р.				Вебплатформа для вивчення мов програмування в ігровій формі				Літ	Аркуш	Аркушів	
Перевірів	Когут В. С.								Н		2	53
									ХПФК			
Н.контр.												
Затвер.												

ВСТУП

У сучасному світі, де цифрові технології стрімко розвиваються, а знання мов програмування стає необхідною складовою професійного розвитку, особливо важливо зробити процес навчання максимально ефективним і доступним. Багато людей стикаються з труднощами під час вивчення програмування: складність матеріалу, відсутність мотивації, нестача практичних завдань і незручна подача інформації. У результаті навчання часто стає нудним і неефективним.

Веб-платформа для вивчення мов програмування є сучасним інструментом, який дозволяє не лише отримувати теоретичні знання, а й одразу застосовувати їх на практиці. Завдяки інтерактивним вправам, тестам та системі миттєвого зворотного зв'язку користувач може швидко засвоювати матеріал, аналізувати свої помилки та покращувати результати. Крім того, впровадження елементів гейміфікації, таких як внутрішньоігрова валюта, система нагород і механіка «вогника», робить процес навчання більш цікавим і мотивуючим.

Замість хаотичного пошуку інформації в різних джерелах, користувач отримує єдине середовище для навчання з чіткою структурою курсів, логічною послідовністю матеріалу та можливістю відстеження власного прогресу.

Актуальність теми:

- підвищення ефективності навчання – поєднання теорії та практики дозволяє краще засвоювати матеріал;
- економія часу – структурований доступ до навчального контенту усуває необхідність пошуку інформації в різних джерелах;
- підвищення мотивації – використання гейміфікації стимулює регулярне навчання;
- зручність контролю знань – автоматична перевірка відповідей і аналіз помилок допомагають швидко оцінити рівень підготовки;
- доступність – веб-платформа може використовуватись у будь-який час і з будь-якого пристрою.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		3

1 ЗАГАЛЬНИЙ РОЗДІЛ

1.1 Постановка завдання

Розробити веб-сайт для вивчення мов програмування. Реалізувати функціонал реєстрації користувачів із введенням імені, електронної пошти та пароля, а також прив'язку облікового запису до прогресу навчання користувача.

Створити сторінки навчальних завдань, що містять теоретичні матеріали, приклади коду, інтерактивні вправи та тести. Реалізувати можливість вибору курсу навчання залежно від мови програмування, яку користувач бажає вивчати.

Додати систему нагород за проходження рівнів у вигляді внутрішньоігрової валюти, можливість придбання життів за ігрову валюту, механіку «вогника» серії днів безперервного навчання та можливість його відновлення за ігрову валюту.

Передбачити систему миттєвої перевірки відповідей, пояснення допущених помилок і підрахунку правильних відповідей, де розмір нагороди залежить від відсотка успішно виконаних завдань.

Додатково передбачити адміністративну панель доступу, за допомогою якої адміністратор має можливість додавати нові мови програмування, редагувати та наповнювати теоретичні матеріали, а також створювати і налаштовувати тести. Забезпечити розмежування прав доступу між звичайними користувачами та адміністратором.

1.2 Опис вхідної інформації

На етапі проєктування веб-застосунку важливо визначити, які саме дані вводитиме користувач. Це дозволяє закласти основу для реалізації інтерфейсу, логіки обробки інформації та структури бази даних. Основна вхідна інформація умовно поділяється на два типи: дані користувача та навчальні дані, пов'язані з проходженням курсів.

Інформація про користувача включає такі поля: ім'я, яке використовується для персоналізації інтерфейсу; електронна пошта — для реєстрації та входу в систему; пароль — для забезпечення безпеки облікового запису. Додатково може зберігатися інформація про прогрес користувача, така як обрана мова

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		4

програмування, поточний рівень, кількість набраних балів або внутрішньоігрової валюти, а також значення «вогника» серії безперервного навчання.

Щодо навчального процесу, користувач взаємодіє з різними типами даних. До них належать теоретичні матеріали, які містять текстову інформацію та приклади коду; інтерактивні завдання, де користувач вводить відповіді або пише фрагменти коду; а також тести, що складаються з питань і варіантів відповідей. Для кожного завдання зберігається інформація про правильність відповіді, кількість спроб та отриманий результат.

Під час проходження курсів система фіксує дії користувача: завершені уроки, результати тестування, кількість правильних відповідей, отримані нагороди та використану внутрішньоігрову валюту. Це дозволяє формувати статистику навчання та відображати прогрес у зручному вигляді.

Також передбачено, що адміністратор системи матиме можливість вводити та редагувати дані навчального контенту: додавати нові мови програмування, створювати теоретичні матеріали, формувати завдання та налаштовувати тести.

1.3 Опис результуючої інформації

У результаті роботи веб-застосунку для вивчення мов програмування на екран виводиться наступна інформація:

Перед початком роботи користувач повинен пройти реєстрацію та створити обліковий запис. У профілі користувач може переглядати свої дані: ім'я, електронну пошту, а також інформацію про власний прогрес навчання. Дані вводяться вручну та зберігаються у базі даних.

На головній сторінці веб-застосунку відображається список доступних мов програмування або курсів. Користувач може обрати потрібний курс і перейти до навчальних матеріалів. Кожен курс містить структуровані уроки, що включають теоретичну інформацію, приклади коду та інтерактивні завдання.

Користувач має можливість проходити навчальні модулі, виконувати вправи та складати тести. Результати виконання завдань зберігаються у базі даних і відображаються у вигляді прогресу. Для кожного завдання фіксується правильність відповіді, кількість спроб та отримані бали.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		5

У системі передбачено відображення результатів навчання у вигляді статистики, зокрема:

- кількість пройдених уроків;
- відсоток правильних відповідей;
- рівень користувача та кількість отриманої внутрішньоігрової валюти;
- значення «вогника» кількість днів безперервного навчання.

Користувач може переглядати свої досягнення та нагороди на окремій сторінці. Також передбачено можливість отримання додаткових життів за внутрішньоігрову валюту та відновлення серії навчання.

У розділі тестування відображаються питання з варіантами відповідей або поля для введення відповіді. Після проходження тесту користувач отримує миттєвий результат із поясненням помилок.

Адміністративна панель дозволяє керувати вмістом платформи: додавати нові курси, редагувати навчальні матеріали, створювати тести та переглядати інформацію про користувачів.

Вся інформація про користувачів, навчальні матеріали, результати тестування та прогрес зберігається у базі даних на сервері, що забезпечує доступ до даних з різних пристроїв.

Таким чином, кінцевий користувач отримує зручний інструмент для навчання програмування, може проходити курси, виконувати завдання, відстежувати свій прогрес і покращувати рівень знань у структурованому та інтерактивному середовищі.

1.4 Стадії та етапи розробки задачі

Розробка веб-застосунку для вивчення мов програмування здійснюється відповідно до класичних етапів життєвого циклу програмного забезпечення. Як основну обрано каскадну модель розробки, яка передбачає послідовне виконання всіх етапів — від аналізу вимог до супроводу готового продукту.

На етапі аналізу проведено збір і формалізацію вимог до майбутнього веб-застосунку. Визначено основні функціональні можливості:

- реєстрація та авторизація користувачів;

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		6

- реалізація адміністративної панелі для керування контентом;
- вибір мови програмування для навчання;
- перегляд теоретичних матеріалів;
- виконання інтерактивних завдань і тестів;
- автоматична перевірка відповідей із поясненням помилок;
- нарахування внутрішньоігрової валюти та реалізація системи нагород;
- впровадження механіки «вогника» серії безперервного навчання;
- ведення профілю користувача та відстеження прогресу.

На стадії проектування створено структуру бази даних для збереження інформації про користувачів, навчальні курси, завдання, результати тестування та прогрес. Також розроблено основні сторінки веб-застосунку та налагоджено їхню взаємодію: сторінка реєстрації та входу, головна сторінка з вибором курсів, сторінки навчальних матеріалів, тестування, профілю користувача та адміністративної панелі.

Особливу увагу приділено зручності користування та інтуїтивній навігації інтерфейсу. Створено макети сторінок із сучасним дизайном, що забезпечує комфортну взаємодію користувача з системою.

На етапі реалізації передбачено розробку функціоналу реєстрації та авторизації користувачів із збереженням даних у базі. Планується впровадження механізму відображення навчальних матеріалів, виконання інтерактивних завдань і проходження тестів із миттєвою перевіркою результатів.

Також реалізується система гейміфікації, що включає нарахування внутрішньоігрової валюти, відображення досягнень і підтримку серії безперервного навчання. Окрема увага приділяється розробці адміністративної панелі, яка дозволяє додавати нові курси, редагувати матеріали та створювати тести.

Передбачено динамічне відображення даних на сторінках веб-застосунку для забезпечення зручності користування, а збереження та обробка інформації здійснюватиметься на сервері.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		7

На стадії тестування веб-застосунків проходить перевірку коректності основних функцій, зокрема реєстрації користувачів, відображення навчальних матеріалів, виконання завдань і підрахунку результатів. Особливу увагу приділено обробці помилок під час введення даних і перевірки відповідей, що забезпечує стабільну роботу системи.

На етапі впровадження веб-застосунків готується до використання користувачами через браузер. Передбачено забезпечення доступності системи та коректної роботи на різних пристроях.

Після впровадження планується подальший розвиток платформи, що включає розширення функціоналу, оптимізацію роботи системи, вдосконалення інтерфейсу користувача та виправлення виявлених помилок. Також буде забезпечено надійність зберігання даних і стабільність роботи веб-застосунку.

1.5 Опис існуючих методів та рішень

Перед тим як приступити до створення веб-застосунку Codelingo, було проведено детальний аналіз існуючих рішень на ринку з метою виявлення їхніх сильних і слабких сторін. Аналіз охоплював вибір архітектурного підходу, мов програмування, середовища розробки, а також визначення ключових функціональних можливостей, які забезпечать конкурентоспроможність продукту.

Одним із найбільш відомих прикладів є платформа Duolingo, яка використовує ігровий підхід до навчання. У ній реалізовано систему рівнів, нагород, серій навчання «streak», а також інтерактивні вправи. Саме ці ідеї стали основою для розробки Codelingo, який орієнтований на вивчення мов програмування.

Також було проаналізовано платформу SoloLearn, яка надає можливість проходження коротких інтерактивних уроків із програмування. Основною перевагою даного сервісу є доступність матеріалів для початківців та зручний мобільний інтерфейс. Проте система має обмежені можливості практичної роботи з кодом та недостатньо глибоке пояснення складних тем.

Ще одним популярним рішенням є Codecademy, яка орієнтована на практичне навчання шляхом написання коду безпосередньо у браузері. Платформа

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		8

підтримує велику кількість мов програмування та має інтерактивне середовище виконання коду. Незважаючи на широкий функціонал, значна частина можливостей доступна лише за платною підпискою, що обмежує доступність сервісу для користувачів.

Крім того, було розглянуто платформу freeCodeCamp, яка орієнтована на самостійне вивчення веб-розробки. Серед її переваг можна виділити велику кількість практичних завдань та відкритий доступ до матеріалів. Однак інтерфейс системи менш орієнтований на гейміфікацію, що може знижувати мотивацію користувачів під час тривалого навчання.

На відміну від існуючих рішень, Codelingo спрямований на більш глибоке та комплексне опрацювання навчального матеріалу за рахунок поєднання теоретичної та практичної складових навчання. Особлива увага приділяється активній взаємодії користувача із системою, що дозволяє зробити процес навчання більш ефективним, цікавим та зрозумілим навіть для початківців. Основний функціонал реалізується за рахунок:

- інтерактивних вправ із написанням коду;
- миттєвої перевірки відповідей із поясненням помилок;
- розширеної системи відстеження прогресу;
- гейміфікації у вигляді внутрішньоігрової валюти та системи нагород;
- наявності адміністративної панелі для керування навчальним контентом.

Крім того, система орієнтована на підтримку регулярного навчання та підвищення мотивації користувачів за допомогою сучасних механізмів взаємодії та візуального відображення досягнень. Це дозволяє зробити процес навчання більш зручним, зрозумілим та цікавим для користувача, а також сприяє кращому засвоєнню навчального матеріалу. Таким чином, аналіз існуючих платформ дозволив сформулювати чітке бачення майбутнього продукту та визначити напрямки його вдосконалення. Приклад головної сторінки відображено на рисунку 1.1.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		9

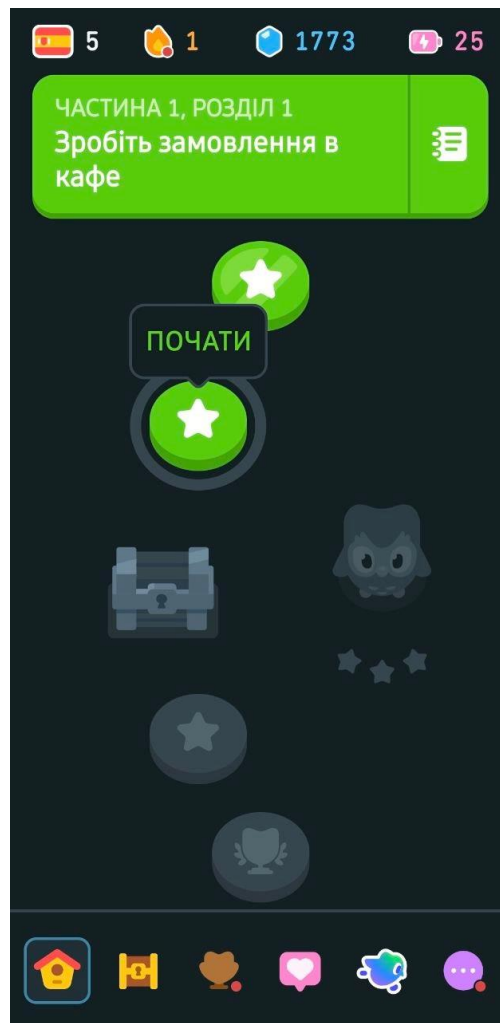


Рисунок 1.1 – Головна сторінка Duolingo

Серед переваг аналогів можна виділити простоту використання, привабливий дизайн та ефективну систему мотивації користувачів. Проте недоліками є обмежений функціонал у безкоштовній версії, недостатня глибина пояснення матеріалу та відсутність можливості гнучкого керування контентом.

Враховуючи ці аспекти, для розробки Codelingo прийнято рішення створити зручний та інтуїтивно зрозумілий інтерфейс, забезпечити доступ до основного функціоналу без обов'язкової платної підписки, а також реалізувати повноцінну систему навчання з детальним аналізом результатів користувача.

Отже, аналіз подібних платформ показав, що існує значний попит на зручні, інтерактивні та доступні сервіси для вивчення програмування. Більшість існуючих рішень або мають обмежений функціонал, або не забезпечують достатньої практичної складової. Тому створення веб-платформи, яка поєднує навчання, практику та гейміфікацію, є доцільним.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		10

Розробка веб-застосунку Codelingo орієнтована на використання через браузер, що забезпечує доступність із різних пристроїв без необхідності встановлення додаткового програмного забезпечення.

Серед середовищ розробки було обрано PhpStorm, яке забезпечує зручну роботу з веб-технологіями, підтримує HTML, CSS, JavaScript та має інструменти для ефективної розробки та налагодження коду.

Для реалізації клієнтської частини використовуються HTML, CSS та JavaScript, що забезпечують створення адаптивного та інтерактивного інтерфейсу. Серверна частина реалізується з використанням мови програмування C#, яка дозволяє ефективно обробляти запити користувачів, працювати з базою даних та реалізовувати бізнес-логіку застосунку.

Для зберігання даних розглядалися різні варіанти, зокрема локальні та серверні бази даних. Враховуючи необхідність збереження прогресу користувачів, результатів тестування та навчального контенту, обрано серверне зберігання даних, що забезпечує доступ до інформації з різних пристроїв і підвищує надійність системи.

У цьому розділі проведено аналіз існуючих рішень і технологій, необхідних для розробки веб-застосунку Codelingo. Обґрунтовано вибір інструментів розробки, мов програмування та підходів до реалізації відповідно до поставлених цілей.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		11

2 РОЗРОБКА ТЕХНІЧНОГО ТА РОБОЧОГО ПРОЕКТУ

2.1 Опис та обґрунтування вибору структури та методу організації вихідних та вхідних даних

У розробці веб-застосунку Codelingo важливо правильно організувати структуру даних, що забезпечує ефективне збереження та обробку як вхідної, так і вихідної інформації. Дані користувачів, навчальні матеріали, результати тестування та прогрес навчання повинні бути чітко структурованими для забезпечення швидкого доступу, аналізу та масштабованості системи.

Вхідні дані включають інформацію, яку вводить користувач, зокрема реєстраційні дані а саме ім'я, електронна пошта, пароль, відповіді на тестові завдання, а також вибір курсів і взаємодію з навчальним контентом. Вихідні дані — це результати роботи системи, такі як оцінка відповідей, статистика навчання, рівень користувача, отримані нагороди та відображення прогресу.

Приклад відображення результатів навчання користувача наведено на рисунку 2.1.

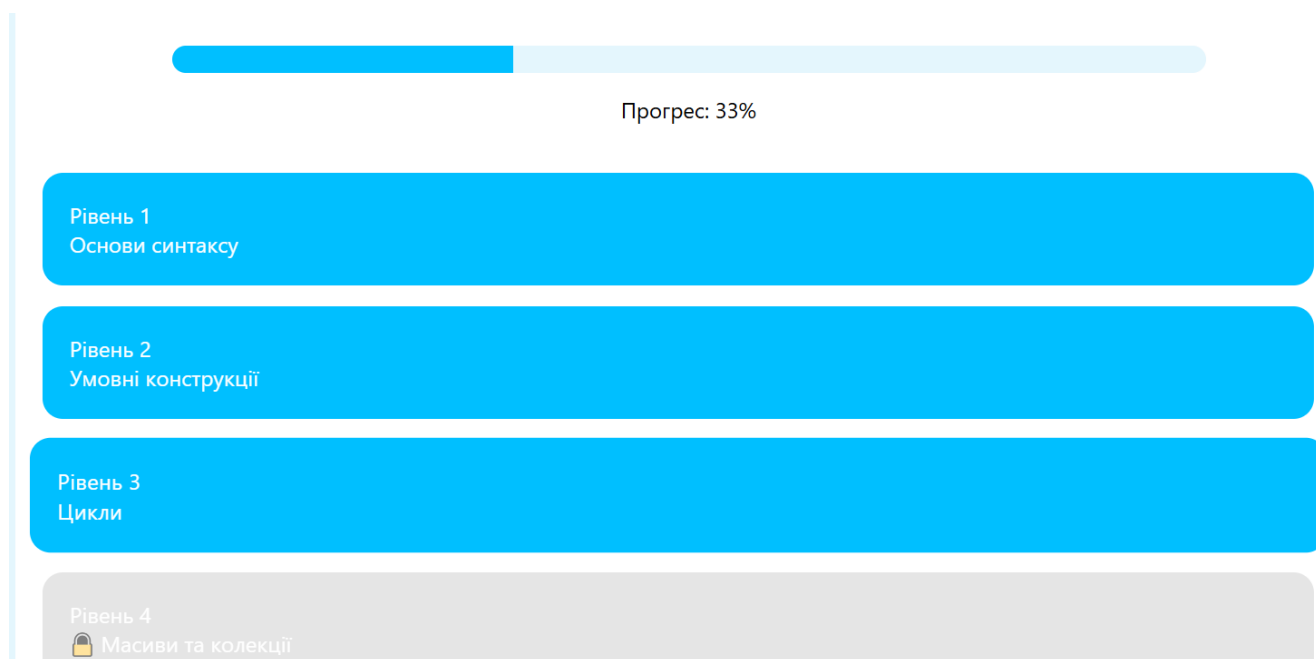


Рисунок 2.1 – Відображення прогресу користувача у веб-застосунку Codelingo

Для роботи з даними використовується серверне зберігання, що дозволяє забезпечити доступ до інформації з різних пристроїв та гарантує надійність збереження даних.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		12

Основні структури даних для організації входу та виходу:

- модель користувача User – зберігає основну інформацію про користувача, а саме: ім'я, електронна пошта, пароль, а також його прогрес, рівень, кількість внутрішньоігрової валюти та значення «вогника»;
- модель курсу Course – містить інформацію про мову програмування, структуру курсу, список уроків і навчальних матеріалів;
- модель уроку Lesson – включає теоретичний матеріал, приклади коду та перелік завдань для виконання;
- модель завдання Question – описує інтерактивні вправи або тестові питання, містить правильні відповіді та правила перевірки;
- модель результатів Result – зберігає інформацію про виконання завдань користувачем, кількість правильних відповідей, набрані бали та статистику.

Схема бази даних включає кілька основних сутностей, таких як Users, Courses, Lessons, Question, Results, де кожна сутність відповідає окремому типу даних.

Така структура дозволяє ефективно організувати зберігання інформації та забезпечує швидкий доступ до необхідних даних.

Завдяки продуманій організації структури даних веб-застосунків Codelingo забезпечує ефективну роботу з великими обсягами інформації, підтримує можливість масштабування та спрощує додавання нового функціоналу в майбутньому.

2.2 Розробка структурної схеми проєкту та об'єктної моделі задачі на основі UML діаграм

Наступний етап розробки включає створення UML-діаграм, які є важливими для візуалізації структури та поведінки системи. Вони допомагають чітко визначити архітектуру проєкту, а також зв'язки між різними його компонентами. Крім того, UML-діаграми відображають застосування об'єктно-орієнтованого підходу, зокрема використання класів, інкапсуляції та взаємодії між об'єктами.

Приклад діаграми варіантів використання наведено на рисунку 2.2.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		13

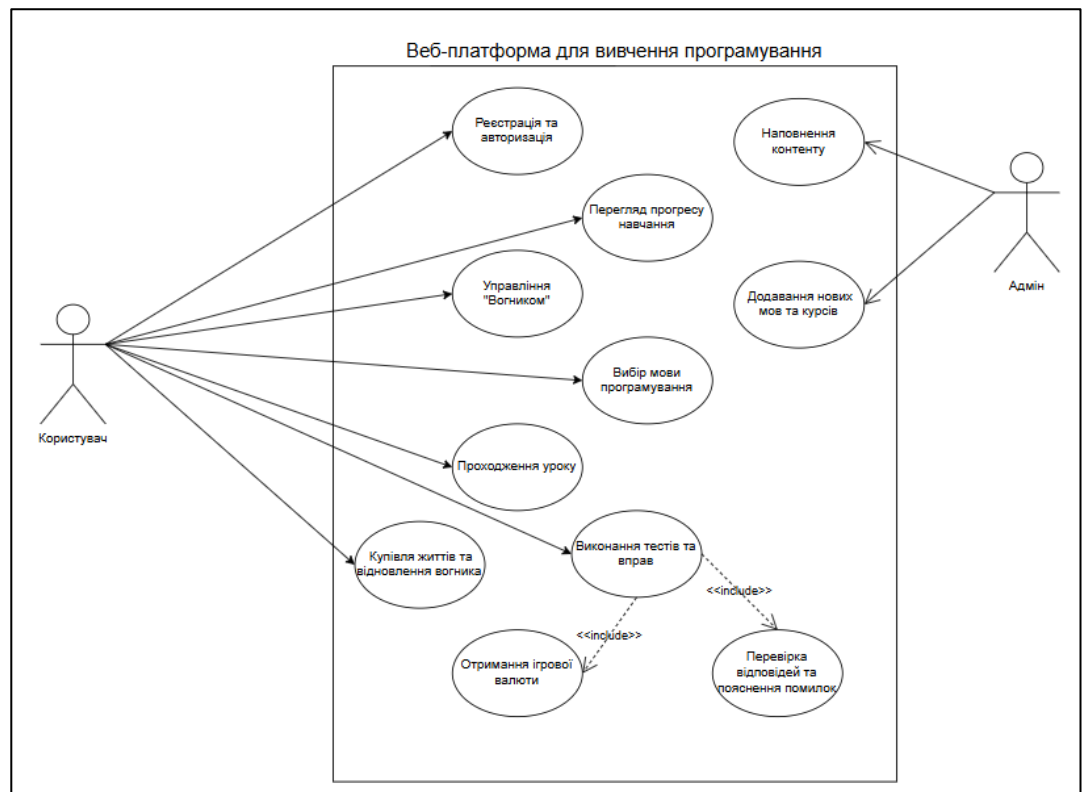


Рисунок 2.2 – Діаграма варіантів використання веб-застосунку Codelingo

На цьому рисунку представлена діаграма варіантів використання, яка відображає основні функціональні можливості системи з точки зору користувача. Центральним актором є «Користувач», який взаємодіє з платформою. Основні сценарії включають реєстрацію, вибір курсу, проходження уроків, виконання завдань, перегляд результатів та отримання нагород.

Також враховано роль адміністратора, який має можливість керувати навчальним контентом: додавати нові курси, редагувати матеріали та створювати тести. Використання зв'язків типу «include» та «extend» дозволяє відобразити залежності між функціями системи.

Приклад діаграми розгортання наведено на рисунку 2.3.

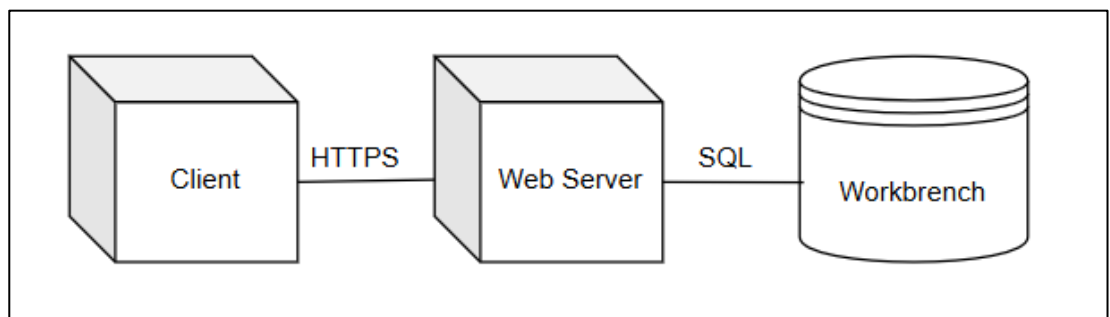


Рисунок 2.3 – Діаграма розгортання веб-застосунку Codelingo

На рисунку представлено фізичну архітектуру системи. Основними компонентами є клієнт браузер користувача та сервер, на якому розміщено веб-застосунок.

Користувач взаємодіє із системою через браузер, використовуючи інтерфейс, створений за допомогою HTML, CSS та JavaScript. Серверна частина, реалізована на C#, відповідає за обробку запитів, логіку роботи та взаємодію з базою даних. У серверній частині застосовано об'єктно-орієнтований підхід, де логіка реалізується через класи, сервіси та контролери.

Між клієнтом і сервером існує мережеве з'єднання, через яке передаються дані. Така архітектура забезпечує масштабованість і доступність системи.

Приклад діаграми активності наведено на рисунку 2.4.

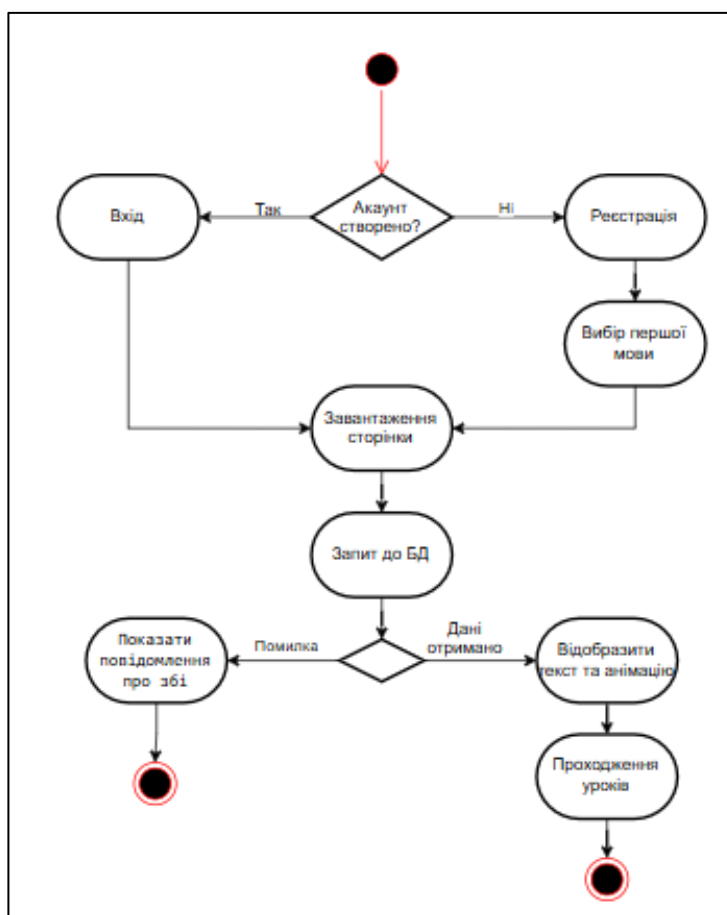


Рисунок 2.4 – Діаграма активності процесу проходження навчання

Діаграма активності демонструє процес взаємодії користувача із системою під час проходження навчального модуля. Спочатку користувач входить у систему, обирає курс і переходить до уроку.

Далі він виконує завдання або проходить тест. Система перевіряє відповідь і, залежно від результату, або нараховує бали та нагороди, або пропонує повторити завдання з поясненням помилок. Логіка перевірки реалізується у відповідних класах і сервісах, що відповідає принципу інкапсуляції — кожен об’єкт виконує свою функцію. Діаграма чітко розділяє дії користувача та системи, що дозволяє зрозуміти логіку роботи застосунку.

Приклад діаграми послідовності наведено на рисунку 2.5.

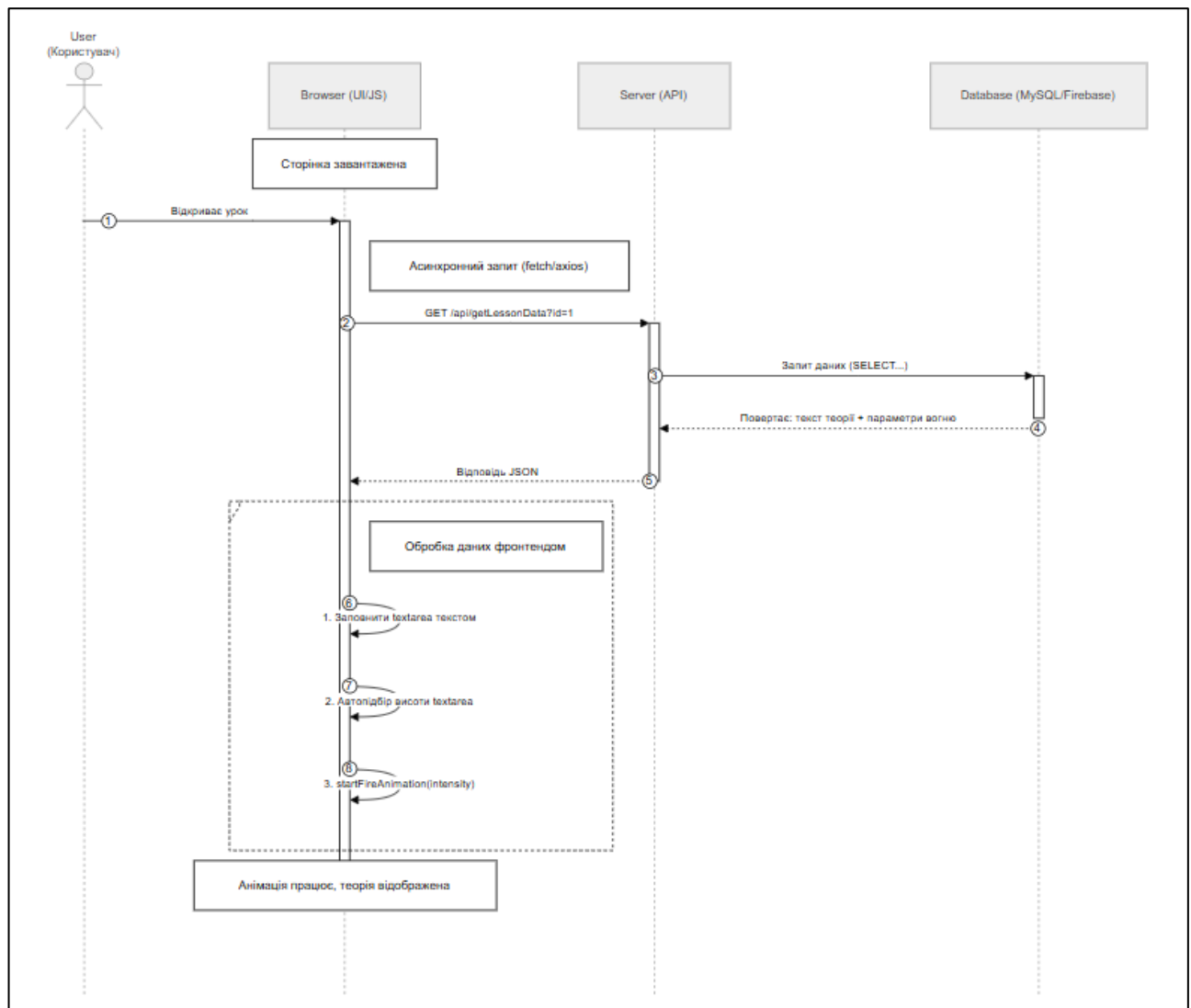


Рисунок 2.5 – Діаграма послідовності взаємодії користувача із системою

Діаграма послідовності відображає порядок взаємодії між користувачем, клієнтською частиною та сервером.

Користувач надсилає запит, наприклад, відповідь на завдання, клієнт передає його на сервер, де відбувається перевірка. Після цього сервер повертає

результат, який відображається користувачу. Взаємодія між компонентами реалізується через виклики методів об'єктів, що є прикладом використання об'єктно-орієнтованої взаємодії.

Приклад діаграми станів наведено на рисунку 2.6.

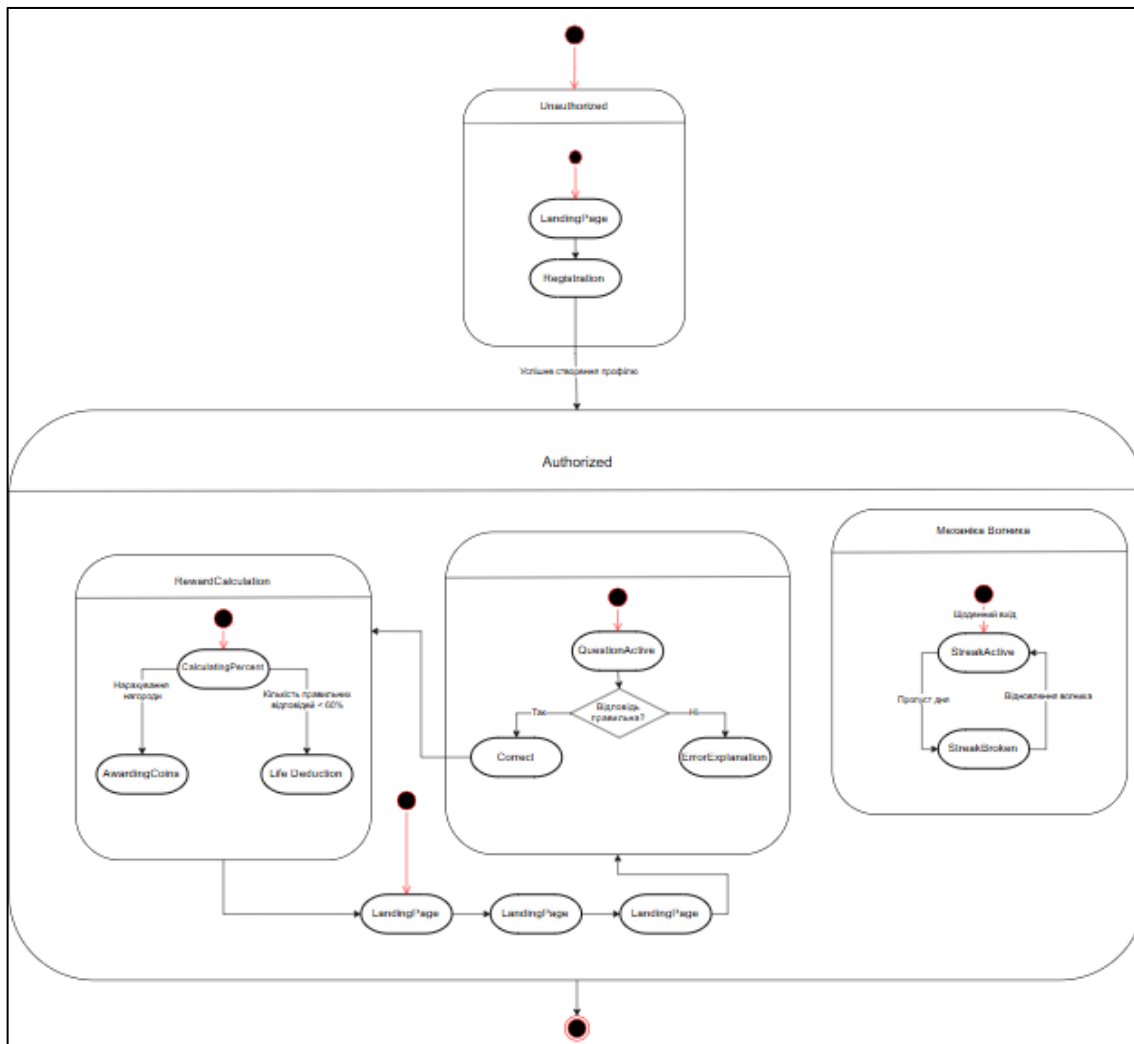


Рисунок 2.6 – Діаграма станів користувача у процесі навчання

Діаграма станів виступає ключовим інструментом для детального опису динаміки поведінки користувача під час його безпосередньої взаємодії з функціоналом системи. Вона дозволяє наочно відобразити послідовність переходу між такими етапами, як реєстрація в базі даних, авторизація профілю, процес вивчення матеріалів курсу, а також виконання практичних вправ із наступною фіксацією отриманих результатів.

Оскільки система гнучко реагує на кожну дію користувача, вона автоматично змінює свій стан, що реалізується через зміну станів об'єктів у

програмі. Це дозволяє ефективно моделювати внутрішню логіку роботи застосунку та забезпечувати стабільність усіх робочих процесів.

Приклад компонентної діаграми наведено на рисунку 2.7.

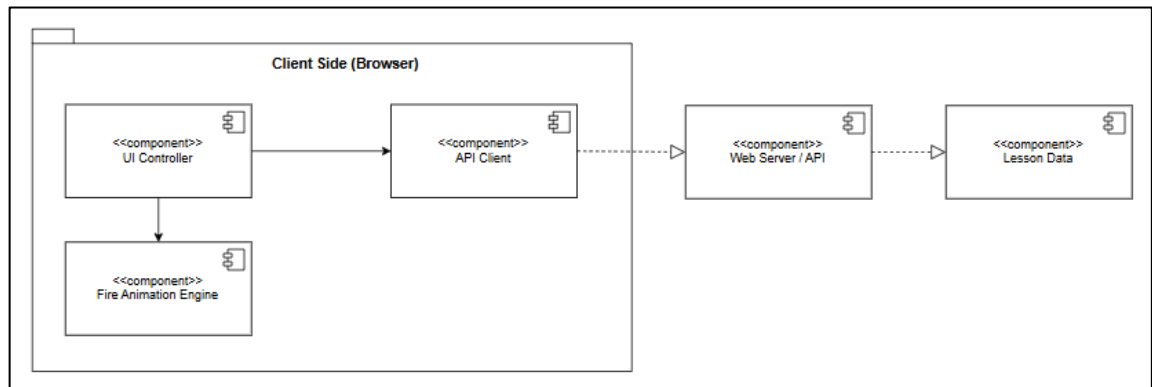


Рисунок 2.7 – Компонентна діаграма веб-застосунку Codelingo

Компонентна діаграма відображає структуру системи та взаємодію її основних частин: клієнтського інтерфейсу, серверної логіки та бази даних. Клієнт взаємодіє із сервером через API. Сервер обробляє запити, використовуючи контролери та бізнес-логіку, сервіси, і звертається до бази даних для збереження та отримання інформації. Такий поділ відповідає принципам об'єктно-орієнтованого програмування та сприяє кращій організації коду.

Приклад діаграми взаємодії наведено на рисунку 2.8.

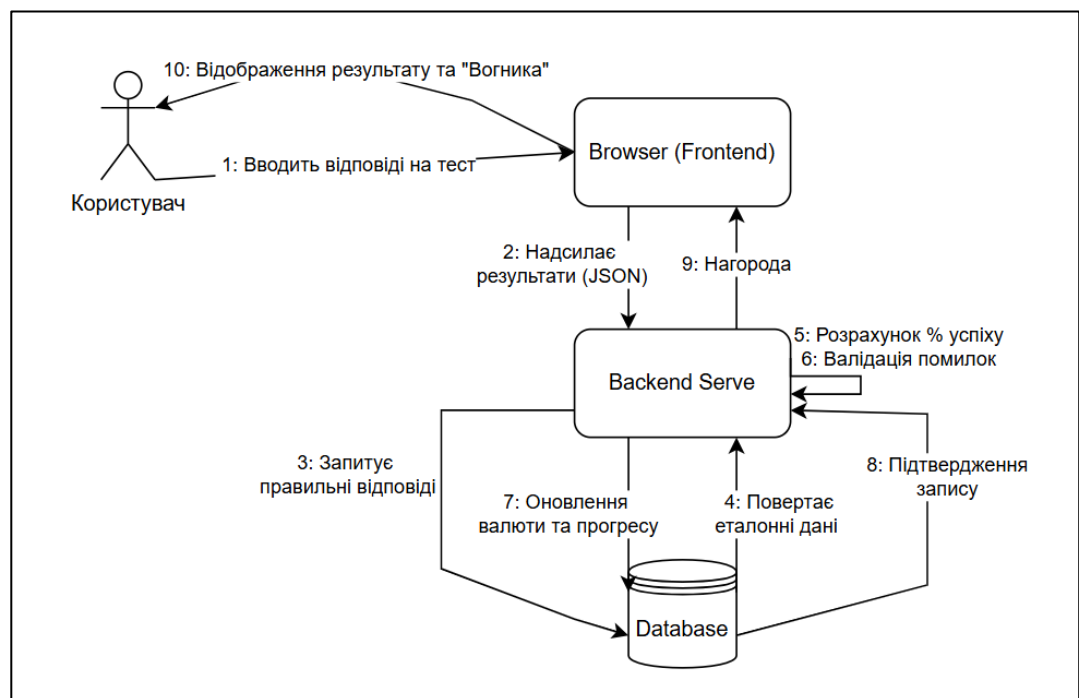


Рисунок 2.8 – Діаграма взаємодії компонентів системи

Дана діаграма демонструє взаємодію між компонентами системи під час виконання конкретного сценарію — наприклад, проходження тесту.

Вона показує, які саме об'єкти беруть участь у процесі та як вони обмінюються даними, що є прикладом реалізації взаємодії між об'єктами в рамках об'єктно-орієнтованого підходу.

Приклад діаграми класів наведено на рисунку 2.9.

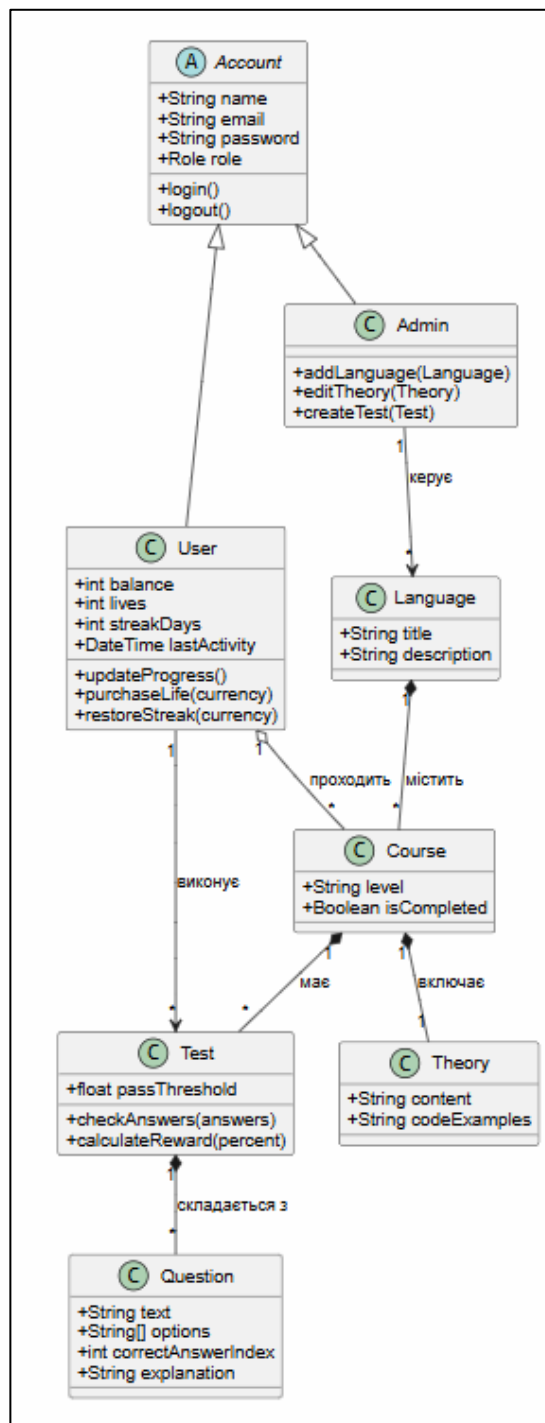


Рисунок 2.9 – Діаграма класів веб-застосунку Codelingo

Діаграма класів відображає основні сутності системи та їх взаємозв'язки. До них належать класи User, Course, Lesson, Question та Result.

Клас User зберігає інформацію про користувача, Course — структуру курсу, Lesson — навчальні матеріали, Question — завдання, а Result — результати виконання.

Ця діаграма демонструє логічну структуру системи та є основою для реалізації програмного коду. Вона також ілюструє застосування принципів об'єктно-орієнтованого програмування, зокрема інкапсуляції – об'єднання даних і методів у класах, а також зв'язків між об'єктами, асоціації та композиції.

2.3 Розробка та опис програмної реалізації та тестування задачі

Розглянемо ключові інструменти, підходи та етапи, які застосовуються під час розробки веб-застосунку для вивчення мов програмування. Для реалізації поставленої задачі використовується сучасний підхід до побудови програмних систем, який передбачає чітке розділення застосунку на клієнтську та серверну частини. Така архітектура дозволяє підвищити гнучкість системи, забезпечити зручність масштабування та спростити процес подальшої підтримки і розвитку програмного продукту.

Особливу роль у розробці відіграє використання принципів об'єктно-орієнтованого програмування – ООП, які дозволяють моделювати систему у вигляді сукупності взаємодіючих об'єктів. Кожен об'єкт має власні дані і стан та поведінку – методи, що забезпечує логічну структурованість та повторне використання коду. Завдяки цьому розробка стає більш керованою, а програмний продукт — більш масштабованим та гнучким до змін.

Серверна частина розробляється у середовищі Visual Studio, яке надає широкий набір інструментів для створення надійних та продуктивних веб-сервісів. На початковому етапі створюється новий проєкт, після чого формується базова структура застосунку, що включає контролери, моделі та сервіси. Контролери відповідають за обробку HTTP-запитів від клієнта, моделі описують структуру даних, які використовуються у системі, а сервіси реалізують бізнес-логіку застосунку.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		20

З точки зору об'єктно-орієнтованого програмування, кожен із зазначених компонентів реалізується у вигляді окремого класу, що відповідає принципу інкапсуляції — об'єднання даних та методів їх обробки в межах однієї сутності. Наприклад, модель користувача може бути представлена у вигляді наступного класу:

```
public class User
{
    public int Id { get; set; }
    public string Email { get; set; }
    public string Password { get; set; }

    public int Coins { get; set; }
    public int Lives { get; set; }

    public void AddCoins(int amount)
    {
        Coins += amount;
    }

    public void LoseLife()
    {
        if (Lives > 0)
            Lives--;
    }
}
```

У наведеному прикладі реалізовано інкапсуляцію: всі дані користувача та методи їх зміни об'єднані в одному класі. Це дозволяє контролювати доступ до стану об'єкта та забезпечує цілісність даних.

Початкова структура серверного проєкту включає основні каталоги та файли, необхідні для організації логіки роботи веб-застосунку. У ній чітко простежується розподіл відповідальності між компонентами, що є важливим принципом сучасної розробки програмного забезпечення. Крім того, така структура відповідає принципу єдиної відповідальності, згідно з яким кожен клас виконує лише одну функцію. Приклад структури серверної частини проєкту наведено на рисунку 2.10.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		21

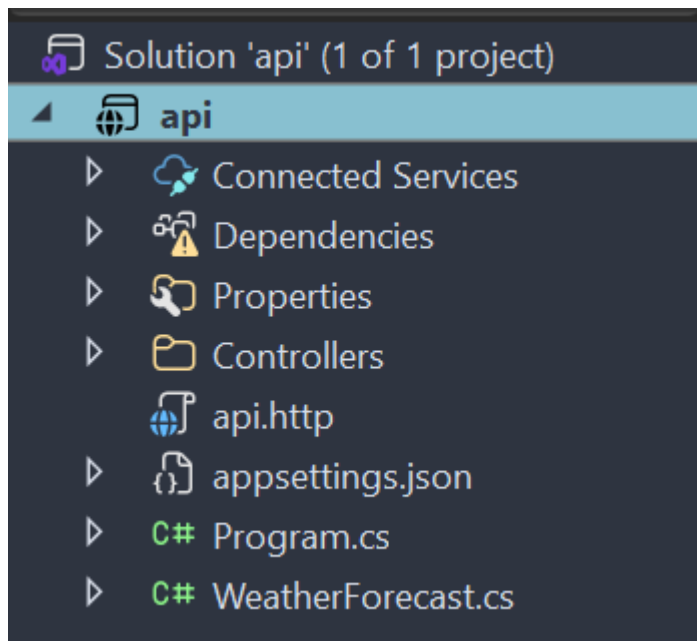


Рисунок 2.10 – Початкова структура серверного проєкту у Visual Studio

У межах серверної частини реалізовано модулі, що відповідають за різні аспекти функціонування системи. Зокрема, модуль користувачів забезпечує реєстрацію та авторизацію. Для реалізації цих функцій використовується сервісний підхід, при якому бізнес-логіка виноситься в окремі класи:

```
public class AuthService
{
    public string Register(string email, string password)
    {
        // логіка реєстрації
        return "User registered";
    }

    public string Login(string email, string password)
    {
        // логіка авторизації
        return "Token";
    }
}
```

Такий підхід демонструє використання абстракції — складна логіка приховується за простими методами, що викликаються з контролера. Крім того, це дозволяє легко змінювати реалізацію без впливу на інші частини системи.

Модуль навчальних курсів відповідає за надання доступу до теоретичних матеріалів, прикладів програмного коду, інтерактивних вправ і тестових завдань. Його реалізація базується на взаємодії кількох класів:

```
public class Course
{
    public int Id { get; set; }
    public string Name { get; set; }

    public List<Lesson> Lessons { get; set; }
}

public class Lesson
{
    public int Id { get; set; }
    public string Title { get; set; }

    public List<Question> Questions { get; set; }
}
```

У даному випадку використано композицію — об'єкт `Course` містить список `Lesson`, а `Lesson` містить список завдань. Це дозволяє будувати складні структури даних із простих компонентів. Приклад сторінки з навчальними матеріалами наведено на рисунку 2.11.

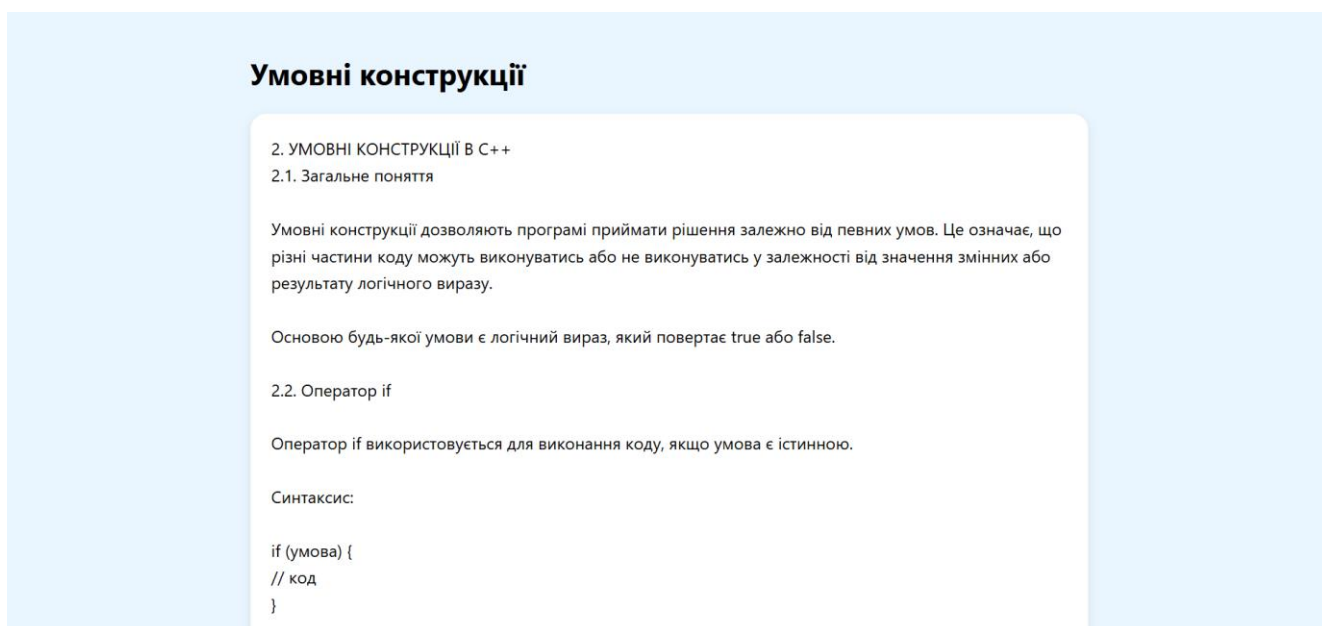


Рисунок 2.11 – Вигляд сторінки навчального курсу у веб-застосунку

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		23

Особливу роль у системі відіграє механізм перевірки знань. Він реалізується через окремі класи та методи, що відповідають за обробку відповідей користувача:

```
public class QuestionkService
{
    public bool CheckAnswer(string userAnswer, string correctAnswer)
    {
        return userAnswer == correctAnswer;
    }
}
```

Це також є прикладом інкапсуляції та абстракції, оскільки вся логіка перевірки зосереджена в одному місці та може бути змінена без впливу на інші компоненти системи. Приклад відображення результатів перевірки наведено на рисунку 2.12.

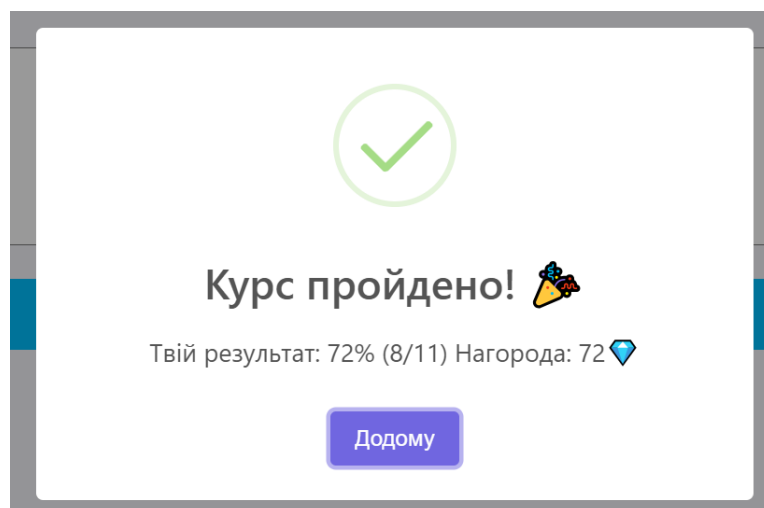


Рисунок 2.12 – Відображення результатів виконання тестових завдань

Окремим компонентом є система гейміфікації. Вона реалізується через властивості та методи класу користувача, а також додаткові сервіси, що керують логікою нарахування винагород. Це дозволяє моделювати поведінку системи як взаємодію об'єктів, що відповідає концепції ООП. Приклад реалізації елементів гейміфікації наведено на рисунку 2.13.

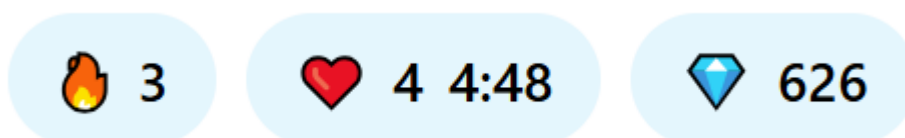


Рисунок 2.13 – Інтерфейс із відображенням гейміфікації: життя, валюта, серія днів

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		24

Фронтенд частина застосунку розробляється у середовищі PhpStorm. Взаємодія між клієнтською та серверною частинами здійснюється через API-запити, що дозволяє реалізувати слабку зв'язаність компонентів системи. Такий підхід є важливим аспектом об'єктно-орієнтованої архітектури, оскільки забезпечує незалежність модулів. Приклад головної сторінки застосунку наведено на рисунку 2.14.

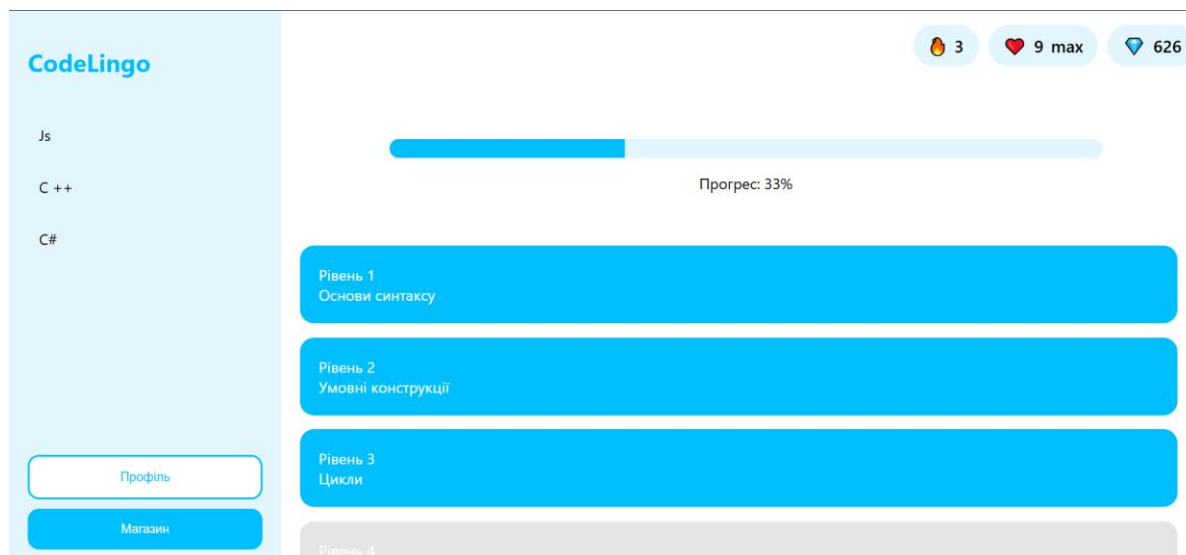


Рисунок 2.14 – Головна сторінка веб-застосунку для навчання

Для зберігання даних у розробленому веб-застосунку використовується база даних, яка виступає ключовим елементом забезпечення стабільної та ефективної роботи всієї системи. Вона дозволяє організовано зберігати інформацію про користувачів, навчальні курси, завдання, результати проходження тестів та інші сутності, що забезпечують функціонування застосунку. Використання бази даних гарантує цілісність, надійність та доступність інформації незалежно від кількості користувачів та інтенсивності роботи системи.

У програмному коді ці дані представлені у вигляді класів, що реалізують концепцію об'єктно-реляційного відображення (ORM). Такий підхід дозволяє встановити відповідність між таблицями бази даних та об'єктами в програмі, де кожен клас відображає окрему сутність предметної області. Наприклад, клас користувача відповідає таблиці користувачів, а клас курсу — таблиці курсів. Це забезпечує більш інтуїтивно зрозумілу роботу з даними, оскільки замість складних SQL-запитів розробник оперує об'єктами та їх властивостями.

Застосування об'єктно-реляційного підходу також тісно пов'язане з принципами об'єктно-орієнтованого програмування, такими як інкапсуляція та абстракція. Дані та методи їх обробки об'єднуються в межах класів, що дозволяє приховати внутрішню реалізацію роботи з базою даних та забезпечити зручний інтерфейс для взаємодії з нею. Крім того, це значно спрощує процес розробки, тестування та подальшого супроводу програмного продукту, оскільки зміни в структурі бази даних можуть бути легко відображені у відповідних класах без суттєвого впливу на інші частини системи.

Адміністративна панель є важливою складовою веб-застосунку та призначена для централізованого керування навчальним контентом і налаштуваннями системи. Вона забезпечує зручний інтерфейс для виконання адміністративних функцій, таких як додавання нових курсів, редагування навчальних матеріалів, створення тестових завдань, а також управління користувачами. Завдяки цьому адміністратор має можливість оперативно оновлювати контент та підтримувати актуальність інформації в системі.

Реалізація адміністративної панелі здійснюється з урахуванням ролей користувачів, що визначають рівень доступу до функціональних можливостей системи. Це дозволяє забезпечити безпеку та захист даних, обмежуючи доступ до критичних операцій лише для авторизованих осіб. Такий підхід може бути реалізований як за допомогою умовних перевірок, так і шляхом створення окремих класів або механізмів, що відповідають за управління правами доступу.

З точки зору об'єктно-орієнтованого програмування, система ролей і прав доступу може розглядатися як окремий рівень абстракції, який визначає поведінку об'єктів залежно від типу користувача. Це дозволяє гнучко налаштовувати функціонал системи та забезпечує можливість подальшого розширення, наприклад, додавання нових ролей або зміну логіки доступу без необхідності суттєвого переписування коду.

Таким чином, використання бази даних у поєднанні з об'єктно-реляційним відображенням та реалізація адміністративної панелі з урахуванням ролей користувачів забезпечують ефективну організацію даних, зручність керування

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		26

системою та високий рівень безпеки. Приклад інтерфейсу адміністративної панелі наведено на рисунку 2.15.

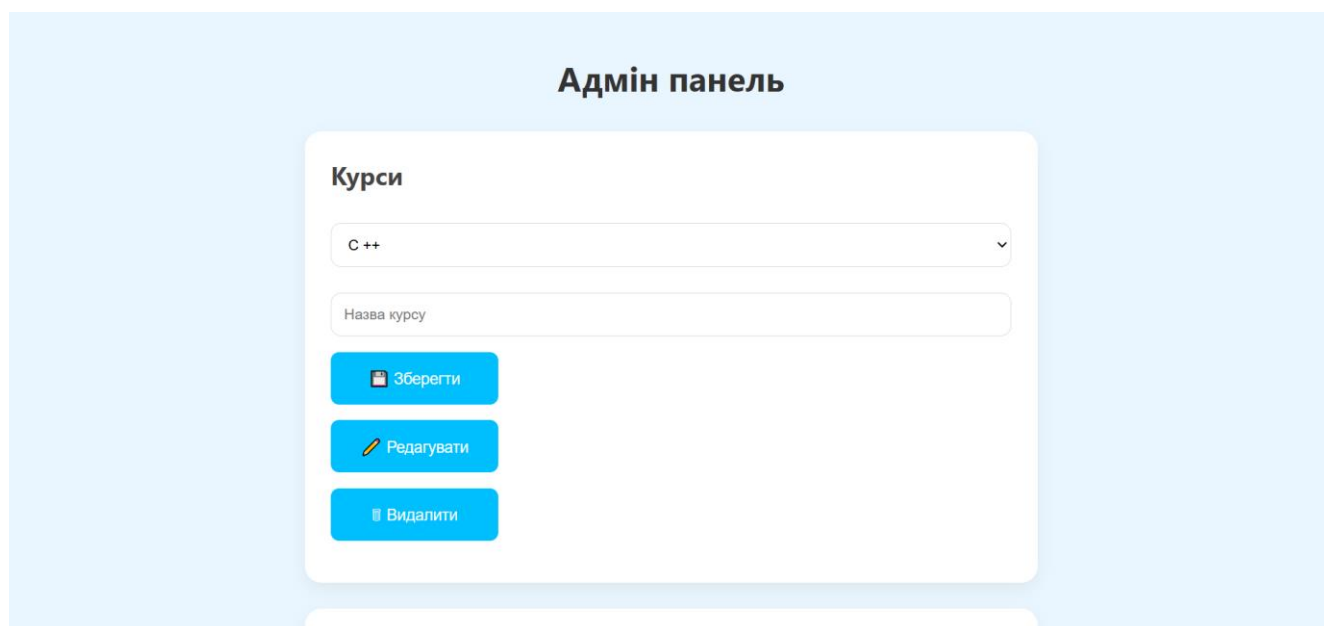


Рисунок 2.15 – Інтерфейс адміністративної панелі керування контентом

Після завершення розробки застосунку проводиться етап тестування. Перевіряється коректність роботи всіх модулів, взаємодія між об'єктами, а також обробка помилок. Приклад відображення прогресу користувача наведено на рисунку 2.16.

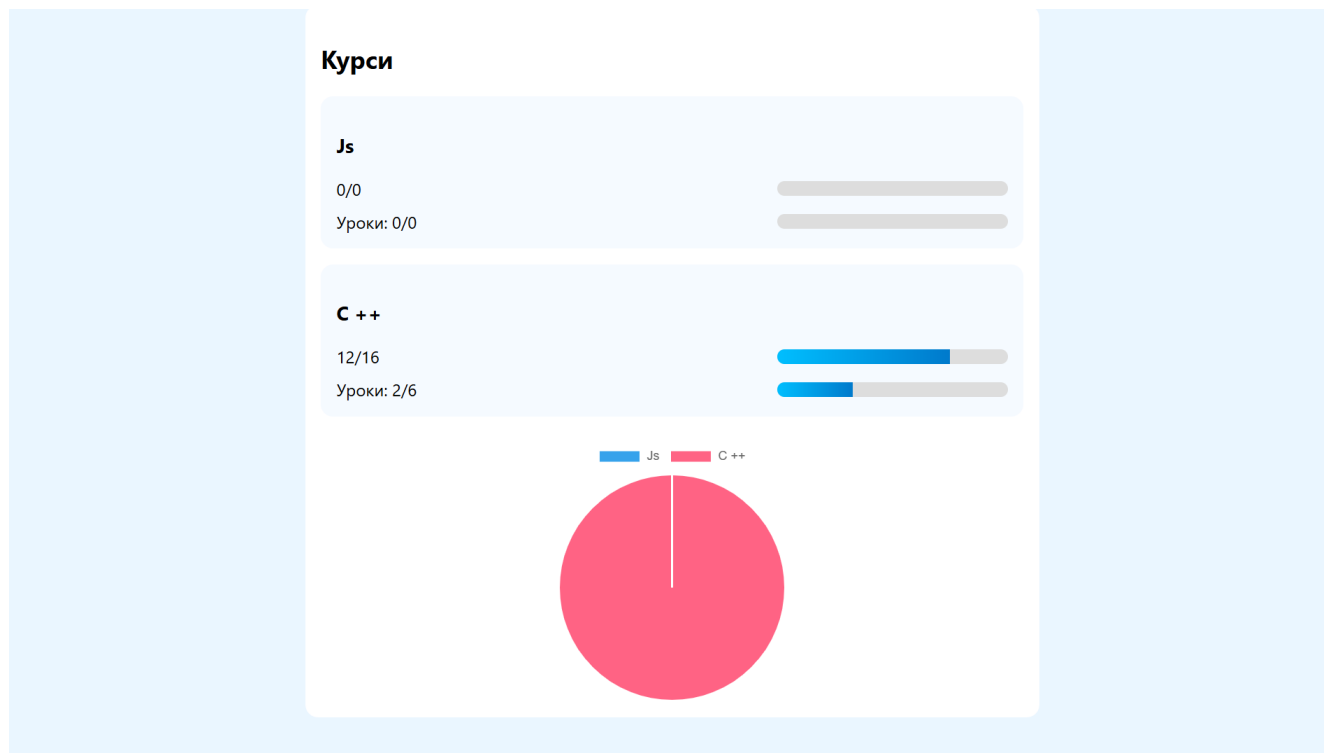


Рисунок 2.16 – Відображення прогресу користувача у веб-застосунку

У результаті виконання даного розділу було реалізовано повноцінний веб-застосунок для вивчення мов програмування. Застосування принципів об'єктно-орієнтованого програмування дозволило створити структуровану, гнучку та масштабовану систему. Використання інкапсуляції, абстракції та композиції забезпечило ефективну організацію коду та спростило його подальшу підтримку. Це також позитивно вплинуло на зручність розробки нових модулів і покращення вже існуючого функціоналу.

Застосування середовищ Visual Studio та PhpStorm дозволило оптимізувати процес розробки та підвищити якість програмного продукту. Обрана архітектура створює передумови для подальшого розвитку системи та розширення її функціональних можливостей. Крім того, реалізований підхід забезпечує зручність у використанні веб-застосунку як для користувачів, так і для подальшої роботи розробника над проєктом.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		28

3 СПЕЦІАЛЬНИЙ РОЗДІЛ

3.1 Інструкція з інсталяції розробленого проекту

Для забезпечення можливості локального запуску та тестування веб-застосунку Codelingo було реалізовано зберігання вихідного коду у репозиторії. Використання репозиторію дозволяє централізовано зберігати файли проєкту, контролювати зміни у коді та спрощує процес встановлення застосунку на інший комп'ютер. Завдяки цьому користувач або розробник може швидко отримати актуальну версію проєкту та розгорнути її у власному середовищі розробки.

Для початку встановлення необхідно перейти до репозиторію проєкту та скопіювати посилання на нього. Приклад сторінки репозиторію наведено на рисунку 3.1.

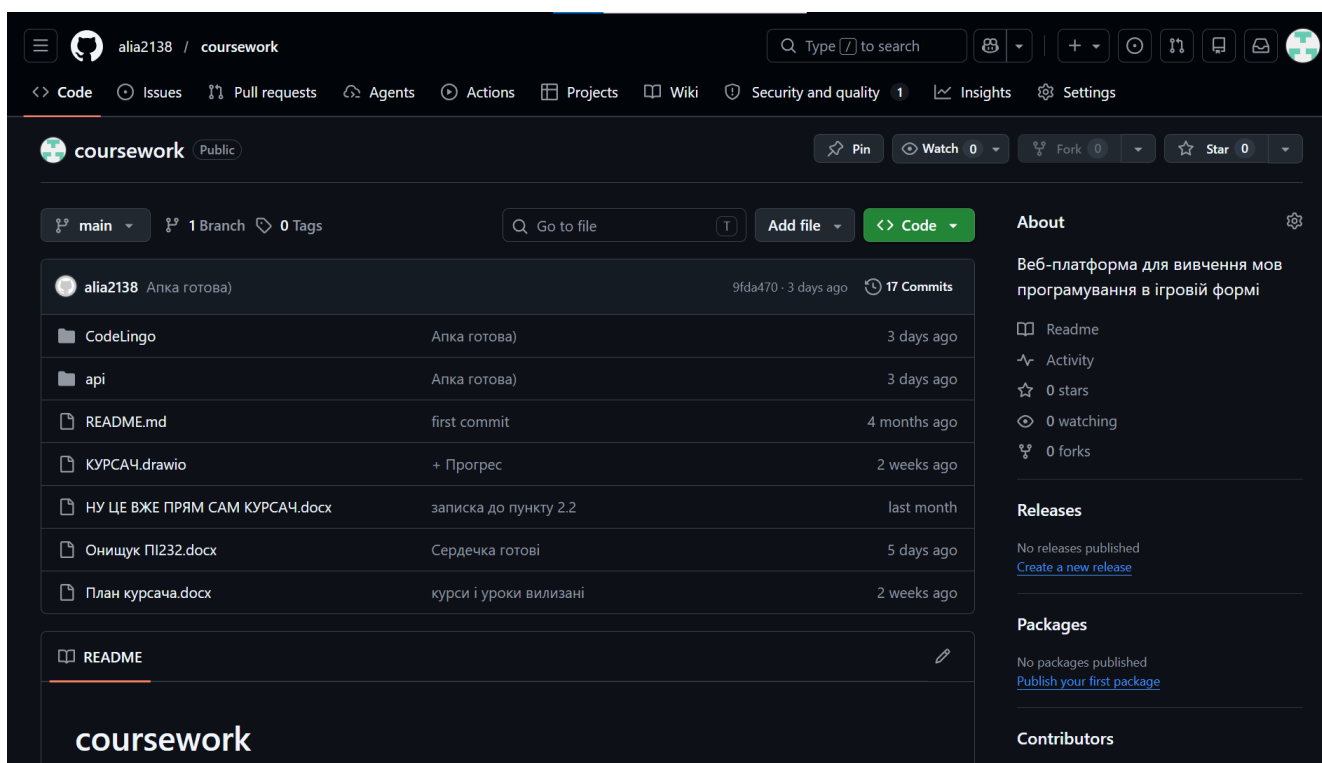


Рисунок 3.1 – Репозиторій веб-застосунку Codelingo

Після цього необхідно відкрити термінал або командний рядок та виконати команду клонування репозиторію:

```
git clone https://github.com/alia2138/coursework.git
```

У результаті виконання команди всі файли проєкту будуть завантажені на локальний комп'ютер у вигляді окремої папки. Приклад процесу клонування репозиторію наведено на рисунку 3.2.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		29

```
User@DESKTOP-NPGQQGE MINGW64 ~/Desktop/codeLingo
$ git clone https://github.com/alia2138/coursework.git
Cloning into 'coursework'...
remote: Enumerating objects: 887, done.
remote: Counting objects: 100% (887/887), done.
remote: Compressing objects: 100% (457/457), done.
remote: Total 887 (delta 442), reused 805 (delta 360), pack-reused 0 (from 0)
Receiving objects: 100% (887/887), 13.65 MiB | 1.02 MiB/s, done.
Resolving deltas: 100% (442/442), done.
```

Рисунок 3.3 – Процес клонування репозиторію

Після завершення клонування необхідно перейти до папки проекту та відкрити серверну частину у середовищі Visual Studio. Для цього потрібно відкрити файл рішення з розширенням .sln. Приклад відкриття проекту у Visual Studio наведено на рисунку 3.4.

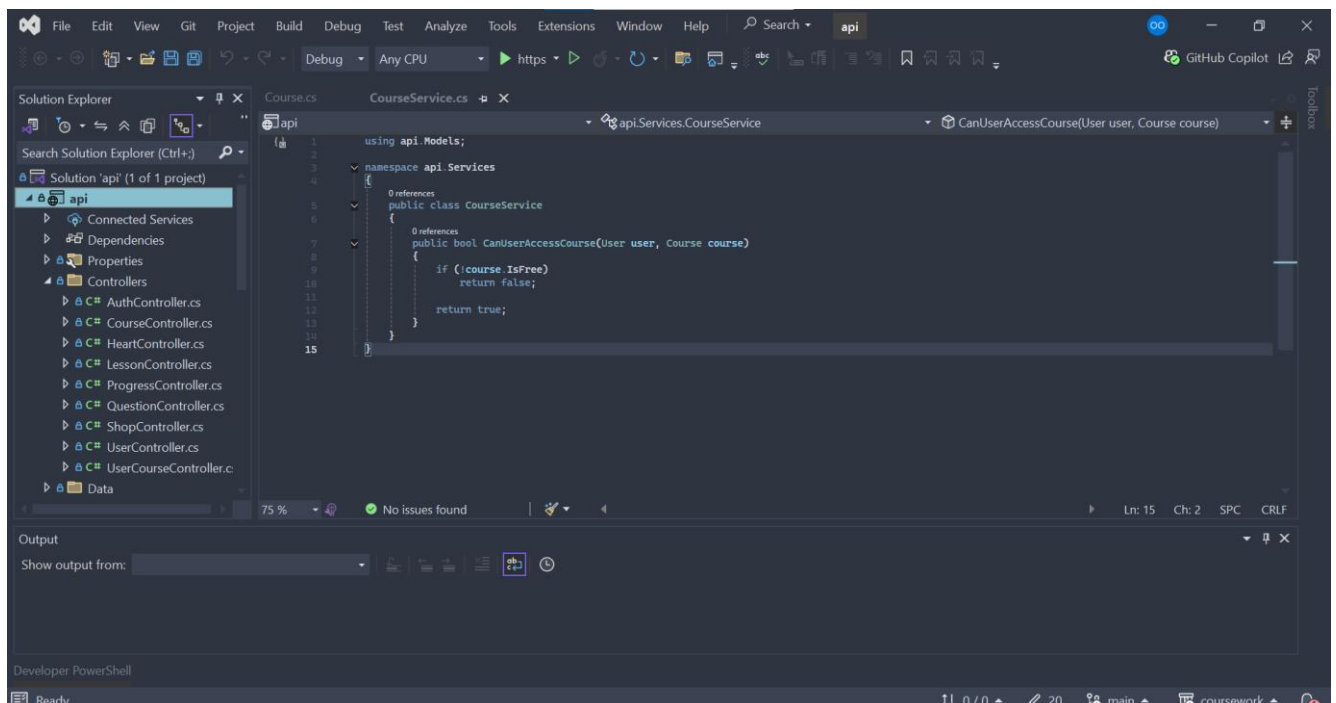


Рисунок 3.4 – Відкриття серверної частини проекту у Visual Studio

Перед запуском застосунку необхідно перевірити наявність установленної платформи .NET SDK, оскільки серверна частина застосунку реалізована з використанням технології ASP.NET Core. Для перевірки версії .NET використовується команда:

```
dotnet --version
```

Якщо платформа не встановлена, її необхідно завантажити та встановити з офіційного сайту Microsoft. Після встановлення середовища розробки та необхідних компонентів можна переходити до запуску серверної частини проекту.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк. 30
Вим.	Арк.	№ докум.	Підпис	Дата		

Наступним етапом є встановлення залежностей проєкту. У більшості випадків Visual Studio автоматично відновлює необхідні пакети NuGet після відкриття рішення. У разі необхідності це можна виконати вручну за допомогою команди:

```
dotnet restore
```

Після успішного відновлення залежностей необхідно налаштувати підключення до бази даних. Для цього у файлі конфігурації appsettings.json потрібно вказати рядок підключення до локальної або віддаленої бази даних. Приклад конфігурації наведено на рисунку 3.5.

```
1  {
2    "ConnectionStrings": {
3      "DefaultConnection": "server=localhost;database=codelingo;user=root;password=Alia2138;"
4    }
5  }
```

Рисунок 3.5 – Налаштування рядка підключення до бази даних

Після налаштування бази даних необхідно виконати міграції для автоматичного створення таблиць у базі даних. Для цього використовується команда:

```
dotnet ef database update
```

У результаті виконання цієї команди буде створено структуру бази даних, необхідну для коректної роботи веб-застосунку.

Після завершення налаштування серверної частини можна перейти до запуску застосунку. Для цього використовується кнопка запуску у Visual Studio або команда:

```
dotnet run
```

Після запуску сервер автоматично відкриє локальну адресу веб-застосунку у браузері. Приклад успішного запуску серверної частини наведено на рисунку 3.6.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		31

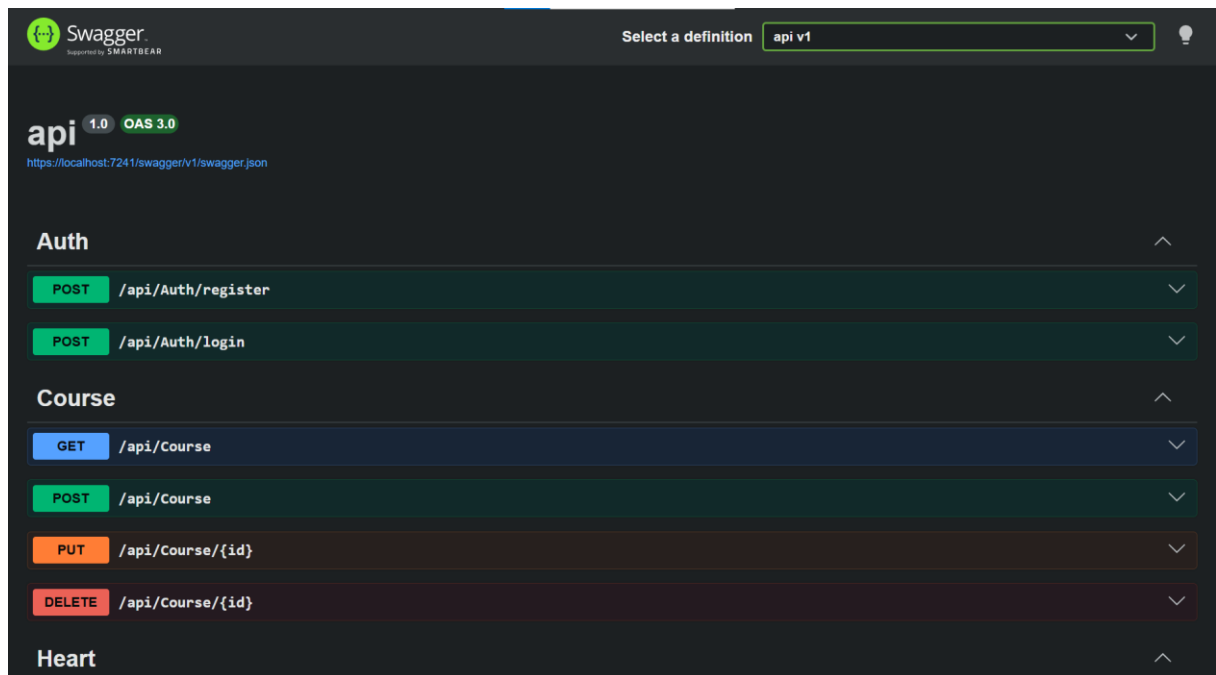


Рисунок 3.6 – Успішний запуск серверної частини застосунку

Фронтенд частина застосунку відкривається у середовищі PhpStorm або будь-якому іншому редакторі коду. Для коректної роботи клієнтської частини необхідно переконатися, що всі файли проєкту завантажені правильно та налаштовано API-запити до серверної частини.

Після запуску клієнтської та серверної частин користувач отримує повністю працездатний веб-застосунок для вивчення мов програмування. Приклад успішного запуску системи наведено на рисунку 3.7.

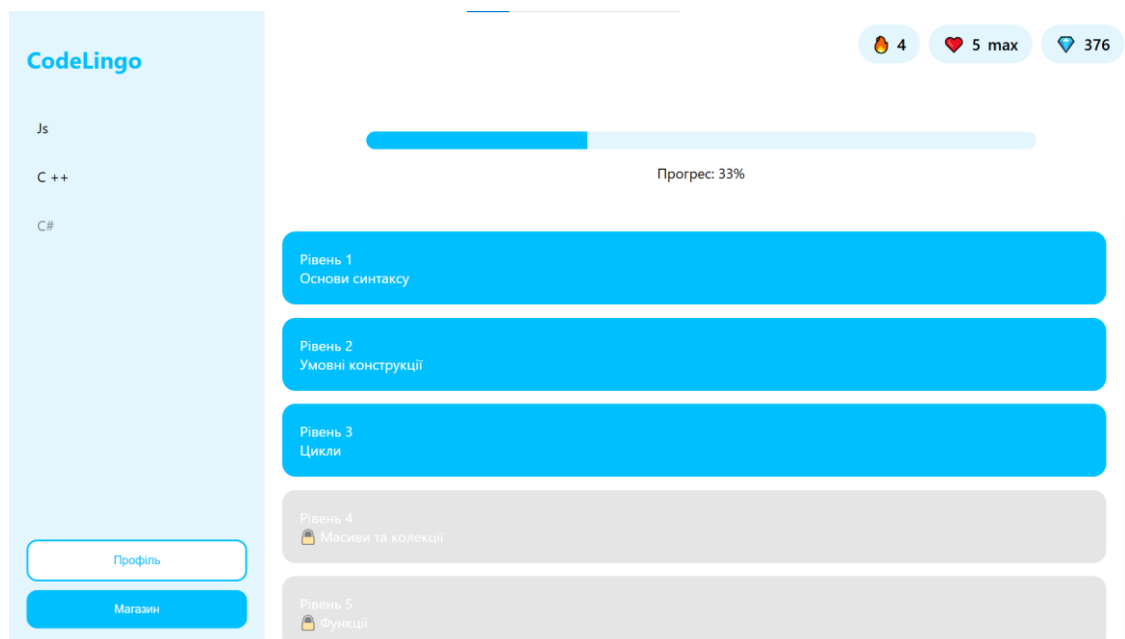


Рисунок 3.7 – Успішний запуск веб-застосунку Codelingo

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		32

Таким чином, процес інсталяції веб-застосунку Codelingo включає клонування репозиторію, встановлення необхідних залежностей, налаштування бази даних та запуск серверної і клієнтської частин проекту. Використання системи контролю версій значно спрощує процес розгортання та подальшої підтримки програмного продукту.

3.2 Інструкція з експлуатації проекту

Після запуску веб-застосунку Codelingo користувач потрапляє на головну сторінку, де йому пропонується виконати вхід до системи або пройти реєстрацію нового облікового запису. Для початку роботи необхідно обрати відповідну дію.

Для створення нового облікового запису користувач переходить на сторінку реєстрації, де вводить необхідні дані: електронну пошту та пароль. Після заповнення форми потрібно натиснути кнопку реєстрації. Приклад сторінки реєстрації наведено на рисунку 3.8.

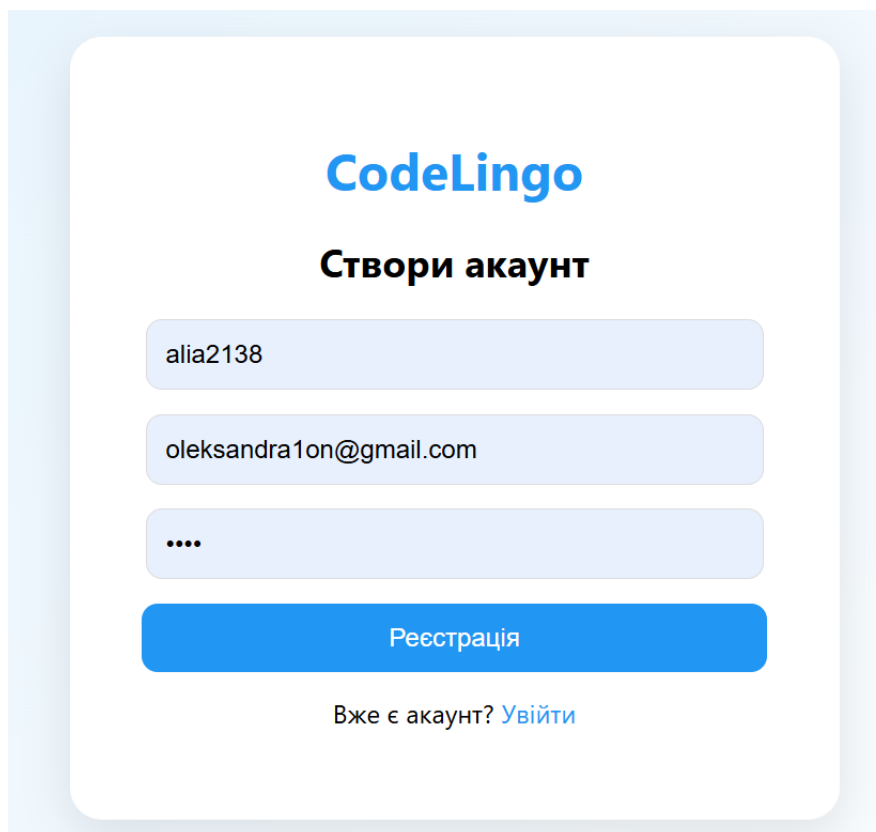


Рисунок 3.8 – Сторінка реєстрації користувача

Після успішної реєстрації користувач може увійти до системи, використовуючи введені дані. У разі правильного введення логіна та пароля

здійснюється перехід до основного інтерфейсу застосунку. Приклад сторінки авторизації наведено на рисунку 3.9.

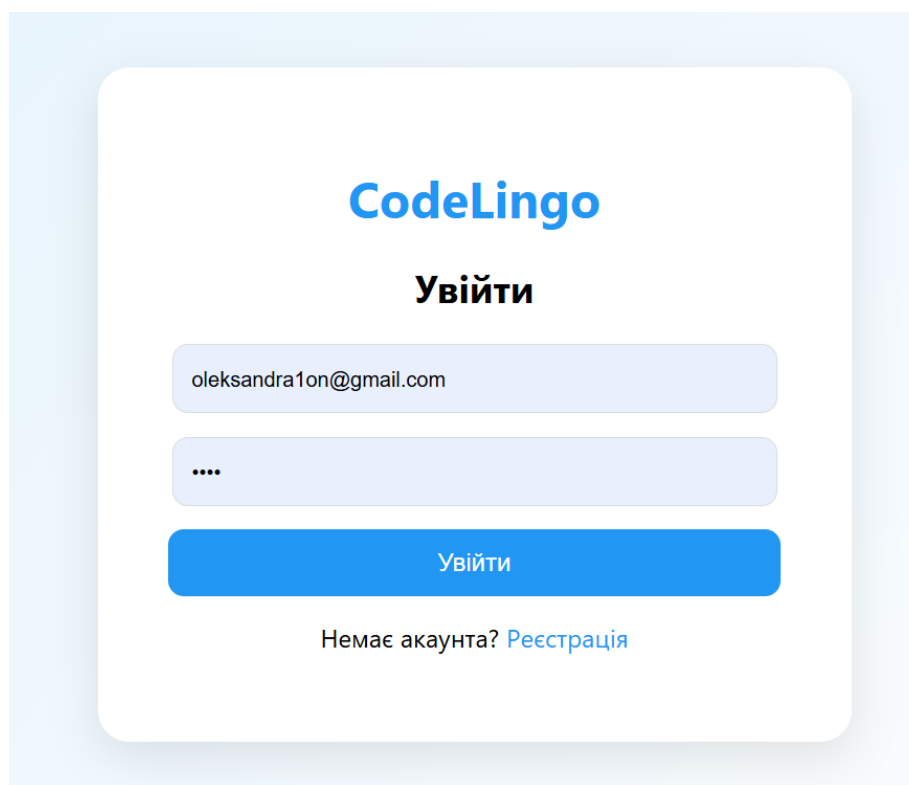


Рисунок 3.9 – Сторінка авторизації користувача

Після входу користувач потрапляє на головну сторінку, де відображаються доступні курси для вивчення. Користувач може обрати будь-який курс, натиснувши на відповідну картку. Приклад вибору курсу наведено на рисунку 3.10.

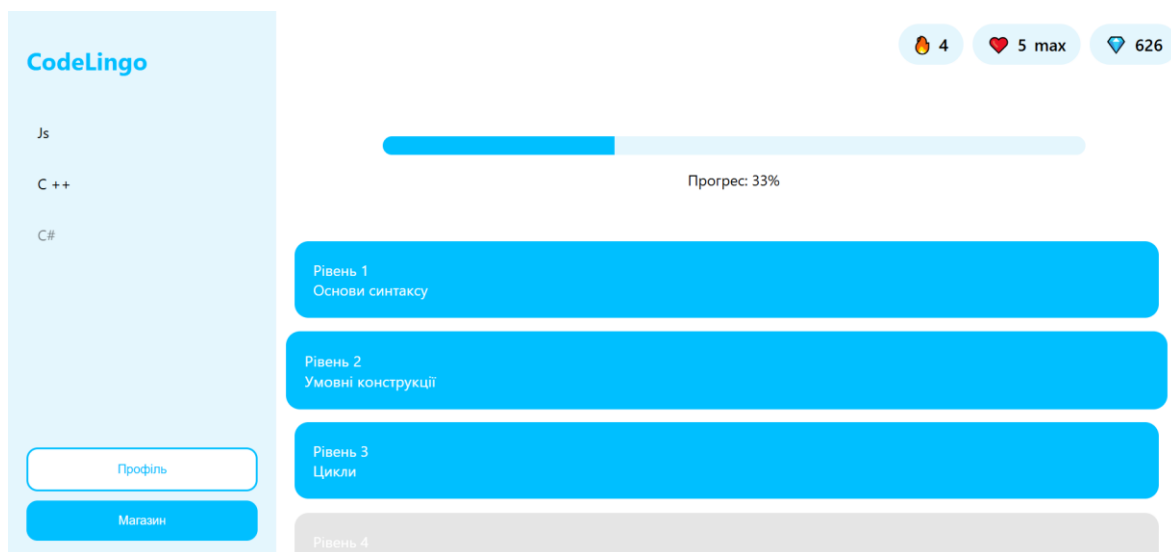


Рисунок 3.10 – Сторінка вибору навчального курсу

Після вибору курсу відкривається сторінка з навчальними матеріалами. Користувач отримує доступ до теоретичної інформації, прикладів коду та пояснень. Приклад сторінки навчального матеріалу наведено на рисунку 3.11.

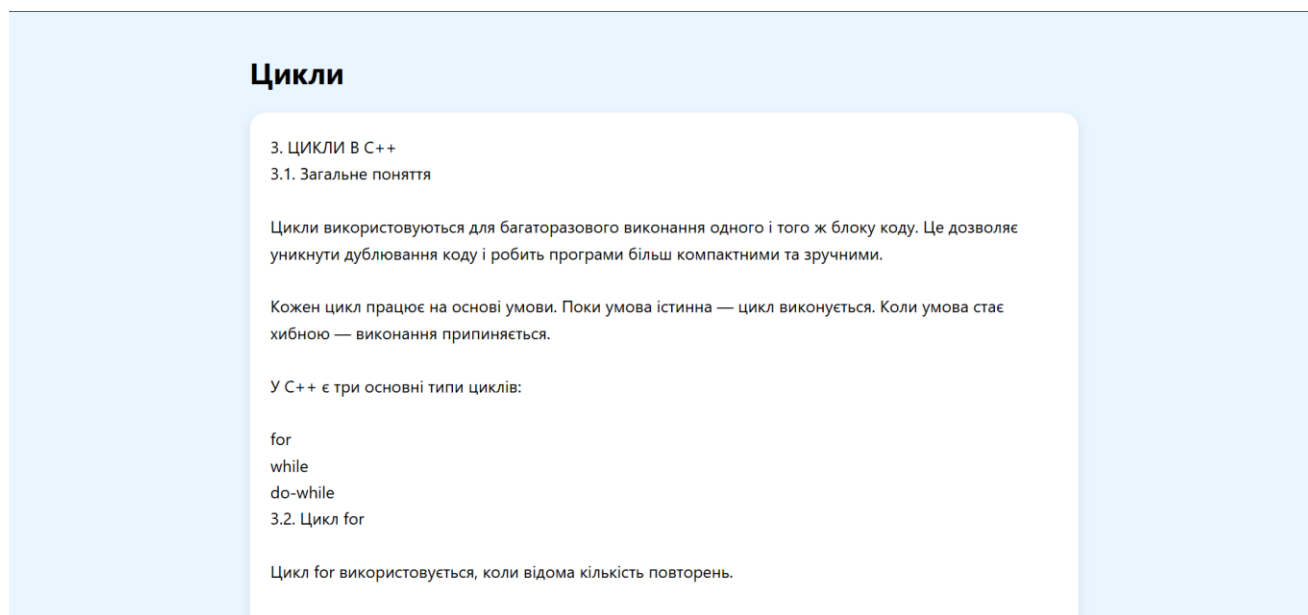


Рисунок 3.11 – Сторінка з навчальним матеріалом

Після ознайомлення з матеріалом користувач переходить до виконання завдань. Для цього необхідно натиснути кнопку переходу до тесту або вправ. Приклад переходу до завдання наведено на рисунку 3.6.

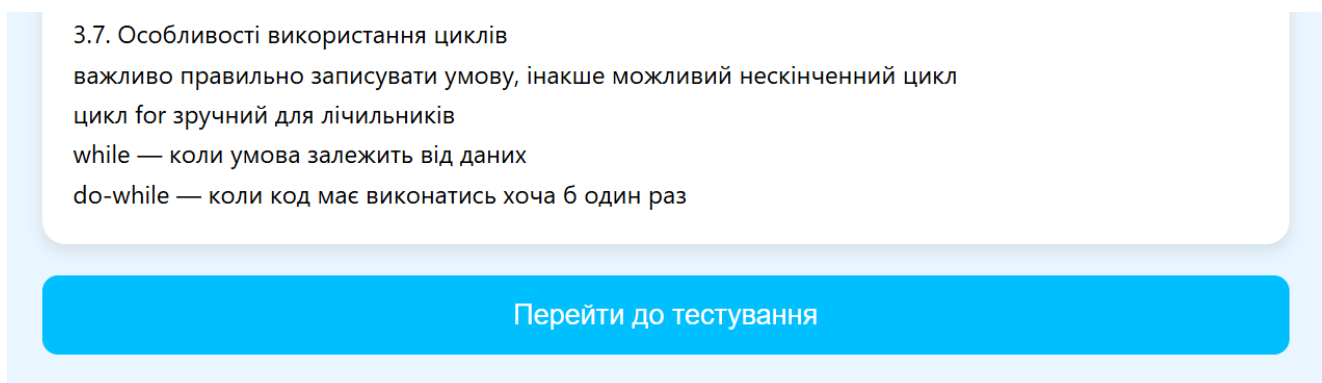


Рисунок 3.12 – Перехід до виконання завдання

На сторінці виконання завдання користувач вводить відповідь або обирає правильний варіант із запропонованих. Після цього необхідно натиснути кнопку перевірки відповіді. Приклад виконання завдання наведено на рисунку 3.12.

Коли перевіряється умова у while?

☐ після виконання

☐ перед виконанням

☐ ніколи

☐ двічі

Перевірити

Рисунок 3.12 – Сторінка виконання тестового завдання

Після відправки відповіді система виконує перевірку та відображає результат. У разі правильної відповіді користувач отримує бали та внутрішньоігрову валюту. Якщо відповідь неправильна — система надає пояснення та пропонує повторити завдання. Приклад відображення результату наведено на рисунку 3.13.

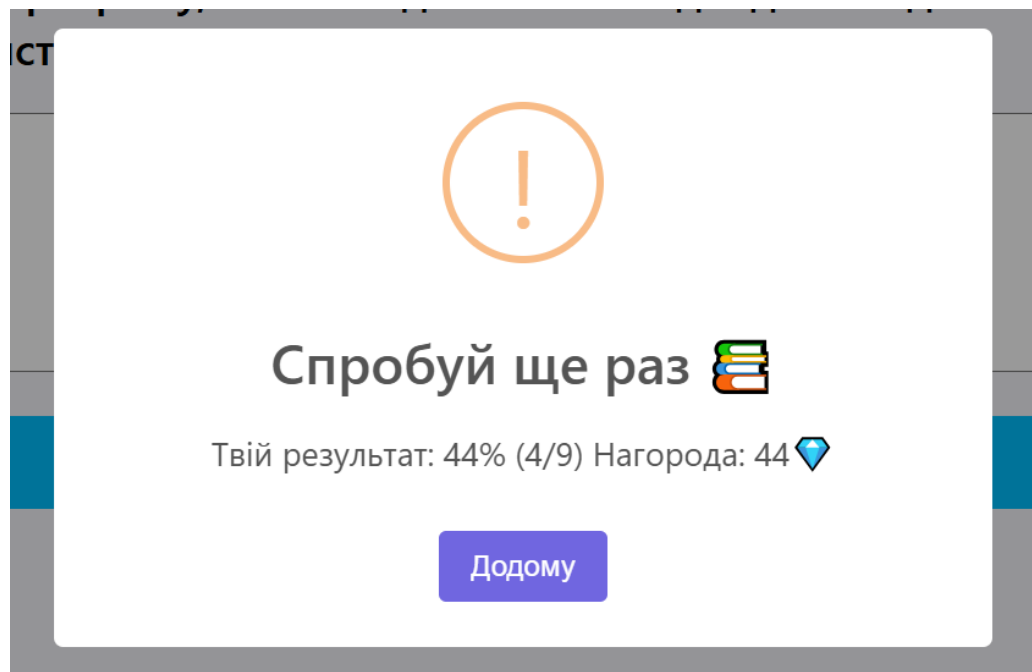


Рисунок 3.13 – Відображення результату перевірки

Під час навчання користувач може спостерігати за своїм прогресом, який включає кількість набраних балів, залишок життів та серію днів навчання. Ці елементи реалізують механізм гейміфікації та підвищують мотивацію до регулярного використання застосунку. Приклад відображення гейміфікації наведено на рисунку 3.14.

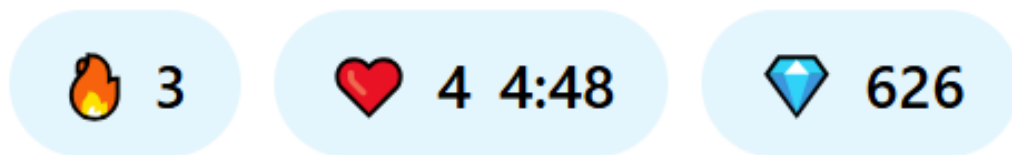


Рисунок 3.14 – Відображення елементів гейміфікації

У разі втрати життів користувач має можливість відновити їх за внутрішньоігровою валюту або дочекатися автоматичного відновлення. Це дозволяє збалансувати процес навчання та зробити його більш інтерактивним.

Користувач також може переглянути свій загальний прогрес навчання. Для цього необхідно перейти до відповідного розділу, де відображається інформація про завершені курси, кількість виконаних завдань та загальний рівень підготовки. Приклад сторінки прогресу наведено на рисунку 3.15.

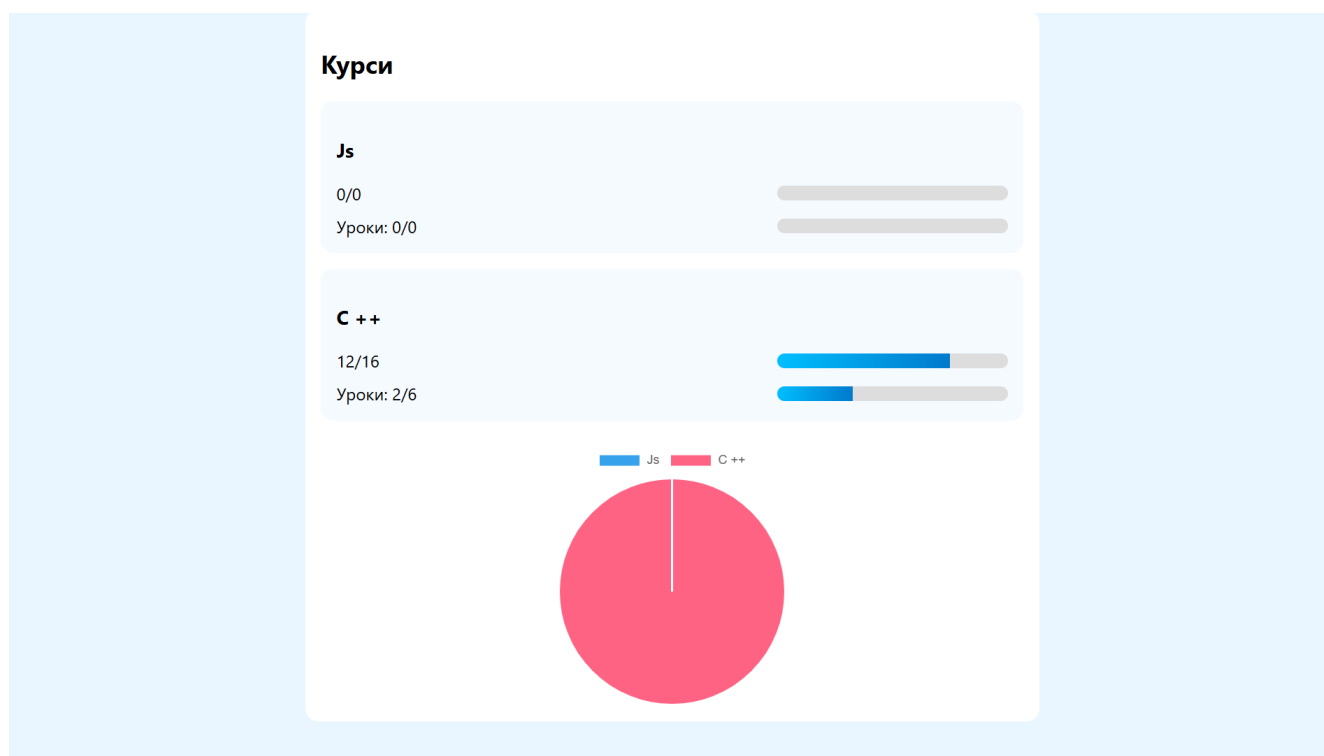


Рисунок 3.15 – Сторінка прогресу користувача

Окремо реалізовано адміністративну панель, доступ до якої має лише адміністратор. У цьому розділі можна додавати нові курси, редагувати навчальні матеріали та створювати тестові завдання. Приклад адміністративної панелі наведено на рисунку 3.16.

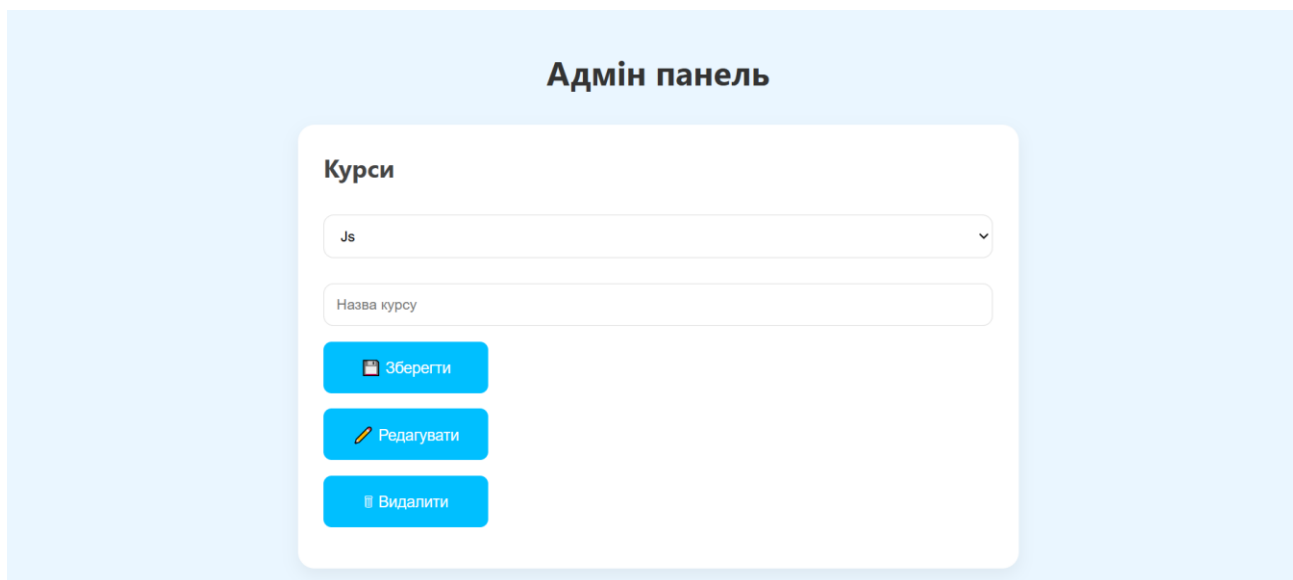


Рисунок 3.16 – Інтерфейс адміністративної панелі

Адміністратор може створювати нові завдання, задавати варіанти відповідей та визначати правильні варіанти. Це дозволяє постійно оновлювати навчальний контент та підтримувати актуальність матеріалів. Приклад створення завдання наведено на рисунку 3.17.

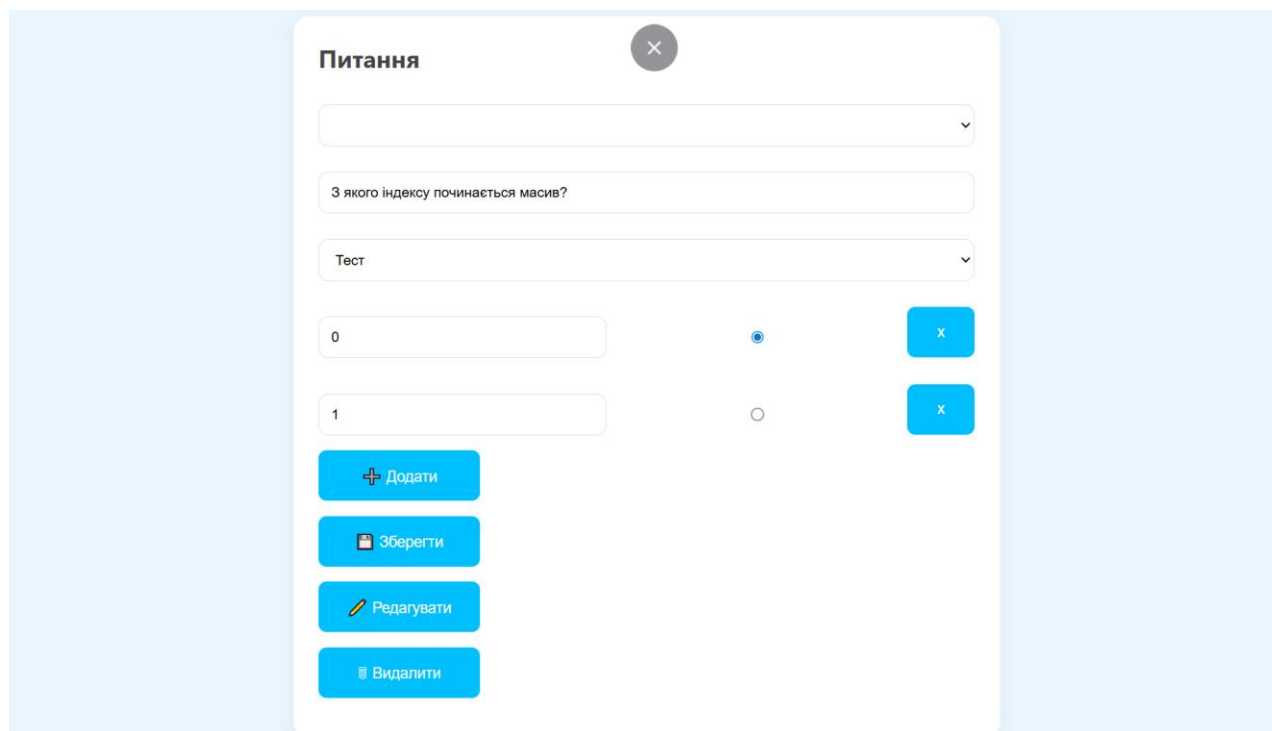


Рисунок 3.17 – Створення тестового завдання

У процесі використання веб-застосунку всі дії користувача автоматично зберігаються у базі даних. До таких дій належать проходження курсів, виконання тестових завдань, отримання нагород та оновлення прогресу навчання. Це дозволяє

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		38

відновлювати прогрес при повторному вході до системи та забезпечує безперервність навчального процесу.

Завдяки автоматичному збереженню даних користувач може продовжити навчання з того місця, на якому він зупинився раніше, без втрати результатів. Крім того, система отримує можливість коректно відображати статистику, рівень прогресу та інші показники активності користувача. Приклад збереження даних наведено на рисунку 3.18.

Id	UserId	LessonId	CorrectAnswers	TotalQuestions	Percentage	IsCompleted	Reward
1	1	6	1	1	100	1	0
2	1	1	6	6	100	1	0
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Рисунок 3.18 – Збереження даних користувача

ВИСНОВКИ

У ході виконання курсової роботи було розроблено веб-застосунок Codelingo для вивчення мов програмування, який поєднує навчальні матеріали, інтерактивні завдання та елементи гейміфікації. Основною метою проєкту було створення зручної та ефективної платформи, що дозволяє користувачам опановувати програмування у структурованій та мотивуючій формі.

У процесі роботи було проаналізовано існуючі рішення у сфері онлайн-навчання, визначено їхні переваги та недоліки, що дало змогу сформуванати власну концепцію застосунку з розширеним функціоналом. На основі цього було сформульовано вимоги до системи та спроектовано її архітектуру.

Реалізація проєкту здійснювалась із використанням сучасних технологій веб-розробки: HTML, CSS, JavaScript для клієнтської частини та C# для серверної логіки. Використання об'єктно-орієнтованого підходу дозволило створити структуровану, модульну та легко масштабовану систему. Додатково було реалізовано об'єктно-реляційне відображення для роботи з базою даних, що забезпечило зручне збереження та обробку інформації.

Особливу увагу приділено функціональності системи: реалізовано реєстрацію та авторизацію користувачів, перегляд навчальних курсів, виконання інтерактивних завдань і тестів, миттєву перевірку відповідей, систему прогресу та нагород, а також механізм «вогника» безперервного навчання. Окремо розроблено адміністративну панель для керування навчальним контентом, що забезпечує гнучкість у наповненні платформи.

У результаті було створено повноцінний веб-застосунок, який поєднує навчальну, практичну та мотиваційну складові. Реалізована система забезпечує зручність використання, доступність з різних пристроїв та можливість подальшого розширення функціоналу.

Отже, поставлені у курсовій роботі завдання виконано повністю, а отриманий програмний продукт може бути використаний як основа для подальшого розвитку освітньої платформи з вивчення програмування.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		40

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Андон Ф.І., Коваль Г.І. Основи проєктування інформаційних систем. – Київ: Видавнича група BVH, 2018.
2. Бородкіна І.Л., Бородкін Г.О. Інженерія програмного забезпечення. – Київ: Центр учбової літератури, 2018.
3. Буч Г. Об'єктно-орієнтований аналіз і проєктування. – Санкт-Петербург: Пітер, 2019. – 720 с.
4. Васильєв О.М. Програмування мовою C#. – Київ: Ліра-К, 2020. – 512с.
5. Глинський Я.М. Практикум з HTML, CSS та JavaScript. – Львів: СПД Глинський, 2021.
6. ДСТУ 3008:2015. Інформація та документація. Звіти у сфері науки і техніки.
7. Кнут Д. Мистецтво програмування. Основні алгоритми. – Москва: Вільямс, 2017.
8. Марченко А.В. Проєктування інформаційних систем. – Київ: ВПЦ «Київський університет», 2016.
9. Мартін Р. Чистий код. Створення і рефакторинг за допомогою Agile. – Харків: Фабула, 2019. – 419 с.
10. Методичні вказівки до виконання курсового проєкту з програмування / ХПК НУ «ЛП». – Хмельницький, 2023.
11. Петрик М., Петрик О. Моделювання програмного забезпечення. – Тернопіль: ТНТУ, 2015.
12. Pressman R. Software Engineering: A Practitioner's Approach. – McGraw-Hill, 2019. – 880 p.
13. Schildt H. C# 10: The Complete Reference. – McGraw-Hill Education, 2022. – 960 p.
14. Sommerville I. Software Engineering. – Pearson Education, 2016. – 816 p.
15. Troelsen A., Japikse P. Pro C# 10 with .NET 6. – Apress, 2022. – 1080 p.
16. <https://learn.microsoft.com/visualstudio/> – документація Visual Studio.
17. <https://learn.microsoft.com/aspnet/core/> – документація ASP.NET Core.

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		41

18. <https://learn.microsoft.com/dotnet/csharp/> – офіційна документація C#.
19. <https://learn.microsoft.com/ef/core/> – документація Entity Framework Core.
20. <https://developer.mozilla.org/docs/Web/JavaScript> – документація JavaScript (MDN).
21. <https://developer.mozilla.org/docs/Web/HTML> – документація HTML (MDN).
22. <https://developer.mozilla.org/docs/Web/CSS> – документація CSS (MDN).
23. <https://docs.github.com/> – документація GitHub.
24. <https://learning.postman.com/docs/> – документація Postman API Platform.
25. <https://swagger.io/docs/> – документація Swagger OpenAPI.
26. <https://docs-neon-psi.vercel.app/> – Документація алгористичної структури даних
27. <https://docs-mai.vercel.app/> - Документація моделювання та аналіз ПЗ

ДОДАТКИ

Додаток А

(Обов'язковий)

Код проекту

Код активності CourseActivity

```
using System.Text.Json.Serialization;

namespace api.Models
{
    public class Course
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public bool IsFree { get; set; }

        [JsonIgnore]
        public List<Lesson>? Lessons { get;
set; } = new List<Lesson>();
        public List<UserCourse>? UserCourses
{ get; set; }
    }
}
namespace api.DTO
{
    public class CreateCourseDTO
    {
        public string Name { get; set; }
    }
}
using api.Data;
using api.DTO;
using api.Models;
using Microsoft.AspNetCore.Mvc;

namespace api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class CourseController :
ControllerBase
    {
        private readonly AppDbContext
_context;

        public CourseController(AppDbContext
context)
        {
            _context = context;
        }

        [HttpGet]
        public IActionResult GetAll()
        {
            return
Ok(_context.Courses.ToList());
        }

        [HttpPost]
```

```
        public IActionResult
AddCourse([FromBody] Course course)
        {
            _context.Courses.Add(course);
            _context.SaveChanges();
            return Ok(course);
        }
        [HttpPut("{id}")]
        public IActionResult
UpdateCourse(int id, [FromBody] Course
updatedCourse)
        {
            var course =
_context.Courses.FirstOrDefault(c => c.Id ==
id);

            if (course == null)
                return NotFound("Курс не
знайдено");

            course.Name =
updatedCourse.Name;

            _context.SaveChanges();

            return Ok(course);
        }
        [HttpDelete("{id}")]
        public IActionResult
DeleteCourse(int id)
        {
            var course =
_context.Courses.FirstOrDefault(c => c.Id ==
id);

            if (course == null)
                return NotFound("Курс не
знайдено");

            _context.Courses.Remove(course);
            _context.SaveChanges();

            return Ok("Курс видалено");
        }
    }
}
using System.Text.Json.Serialization;

namespace api.Models
{
    public class Lesson
    {
        public int Id { get; set; }
        public string Title { get; set; }
```

```

        public string Theory { get; set; }
        public int CourseId { get; set; }
        [JsonIgnore]
        public Course Course { get; set; }
        [JsonIgnore]
        public ICollection<Question>
Questions { get; set; }
    }
}
namespace api.DTO
{
    public class CreateLessonDTO
    {
        public string Title { get; set; }
        public string Theory { get; set; }
        public int CourseId { get; set; }
    }
}
namespace api.DTO
{
    public class CreateQuestionDTO
    {
        public string Text { get; set; }
        public string Type { get; set; }

        public string OptionsJson { get;
set; }
        public string CorrectAnswer { get;
set; }

        public int LessonId { get; set; }
    }
}
using api.Data;
using api.Models;
using api.DTO;
using Microsoft.AspNetCore.Mvc;

namespace api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class LessonController :
ControllerBase
    {
        private readonly ApplicationDbContext
_context;

        public LessonController(ApplicationDbContext
context)
        {
            _context = context;
        }

        [HttpPost]
        public IActionResult
AddLesson([FromBody] CreateLessonDTO dto)
        {
            var lesson = new Lesson
            {
                Title = dto.Title,
                Theory = dto.Theory,
                CourseId = dto.CourseId
            };

```

```

            _context.Lessons.Add(lesson);
            _context.SaveChanges();

            return Ok(lesson);
        }

        // ◆ ОТРИМАТИ УРОКИ ПО КУРСУ
        [HttpGet("{courseId}")]
        public IActionResult GetLessons(int
courseId)
        {
            var lessons = _context.Lessons
                .Where(l => l.CourseId ==
courseId)

                .Select(l => new
                {
                    l.Id,
                    l.Title,
                    l.Theory
                })
                .ToList();

            return Ok(lessons);
        }

        // ◆ ОТРИМАТИ 1 УРОК
        [HttpGet("single/{id}")]
        public IActionResult GetLesson(int
id)
        {
            var lesson = _context.Lessons
                .Where(l => l.Id == id)
                .Select(l => new
                {
                    l.Id,
                    l.Title,
                    l.Theory
                })
                .FirstOrDefault();

            if (lesson == null)
                return NotFound("Урок не
знайдено");

            return Ok(lesson);
        }

        [HttpPut("{id}")]
        public IActionResult
UpdateLesson(int id, [FromBody]
CreateLessonDTO dto)
        {
            var lesson =
_context.Lessons.FirstOrDefault(l => l.Id ==
id);

            if (lesson == null)
                return NotFound("Урок не
знайдено");

            lesson.Title = dto.Title;
            lesson.Theory = dto.Theory;

            _context.SaveChanges();

```

```

        return Ok(lesson);
    }

    [HttpDelete("{id}")]
    public IActionResult
DeleteLesson(int id)
    {
        var lesson =
_context.Lessons.FirstOrDefault(l => l.Id ==
id);

        if (lesson == null)
            return NotFound("Урок не
знайдено");

        _context.Lessons.Remove(lesson);
        _context.SaveChanges();

        return Ok("Урок видалено");
    }
}
using System.Text.Json.Serialization;

namespace api.Models
{
    public class Question
    {
        public int Id { get; set; }

        public string Text { get; set; }

        public string Type { get; set; }

        public string OptionsJson { get;
set; }

        public string CorrectAnswer { get;
set; }

        public int LessonId { get; set; }

        [JsonIgnore]
        public Lesson Lesson { get; set; }
    }
}
using api.Data;
using api.Models;
using api.DTO;
using Microsoft.AspNetCore.Mvc;

namespace api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class QuestionController :
ControllerBase
    {
        private readonly AppDbContext
_context;

        public
QuestionController(AppDbContext context)

```

```

    {
        _context = context;
    }

    [HttpPost]
    public IActionResult
AddQuestion([FromBody] CreateQuestionDTO
dto)
    {
        var lessonExists =
_context.Lessons.Any(l => l.Id ==
dto.LessonId);

        if (!lessonExists)
            return BadRequest("Невірний
LessonId");

        var question = new Question
        {
            Text = dto.Text,
            Type = dto.Type,
            OptionsJson =
dto.OptionsJson,
            CorrectAnswer =
dto.CorrectAnswer,
            LessonId = dto.LessonId
        };

        _context.Questions.Add(question);
        _context.SaveChanges();

        return Ok(question);
    }

    [HttpGet("{lessonId}")]
    public IActionResult
GetQuestions(int lessonId)
    {
        var questions =
_context.Questions
            .Where(q => q.LessonId ==
lessonId)
            .ToList();

        return Ok(questions);
    }

    public class CheckDTO
    {
        public int QuestionId { get;
set; }

        public string Answer { get; set; }
    }

    [HttpGet("get/{id}")]
    public IActionResult GetQuestion(int
id)
    {
        var q =
_context.Questions.FirstOrDefault(x => x.Id
== id);

        if (q == null)
            return NotFound();
    }
}

```

```

        return Ok(q);
    }
    [HttpPost("check")]
    public IActionResult
    CheckAnswer([FromBody] CheckDTO dto)
    {
        var question =
        _context.Questions.FirstOrDefault(q => q.Id
        == dto.QuestionId);

        if (question == null)
            return BadRequest("Питання
не знайдено");

        bool isCorrect = false;

        if (question.Type == "test")
        {
            isCorrect =
            question.CorrectAnswer.Trim().ToLower()
            ==
            dto.Answer.Trim().ToLower();
        }

        if (question.Type == "code")
        {
            string Normalize(string str)
=>
                str.Replace(" ", "")
                .Replace("\n", "")
                .Replace("\r", "")
                .Replace("\t", "")
                .Replace("; ", "")
                .ToLower();

            isCorrect =
            Normalize(question.CorrectAnswer) ==
            Normalize(dto.Answer);
        }

        return Ok(new
        {
            correct = isCorrect,
            correctAnswer =
            question.CorrectAnswer
        });
    }

    [HttpPut("{id}")]

```

```

        public IActionResult
        UpdateQuestion(int id, [FromBody]
        CreateQuestionDTO dto)
        {
            var q =
            _context.Questions.FirstOrDefault(x => x.Id
            == id);

            if (q == null)
                return NotFound("Питання не
знайдено");

            var lessonExists =
            _context.Lessons.Any(l => l.Id ==
            dto.LessonId);

            if (!lessonExists)
                return BadRequest("Невірний
LessonId");

            q.Text = dto.Text;
            q.Type = dto.Type;
            q.OptionsJson = dto.OptionsJson;
            q.CorrectAnswer =
            dto.CorrectAnswer;
            q.LessonId = dto.LessonId;

            _context.SaveChanges();

            return Ok(q);
        }

        [HttpDelete("{id}")]
        public IActionResult
        DeleteQuestion(int id)
        {
            var q =
            _context.Questions.FirstOrDefault(x => x.Id
            == id);

            if (q == null)
                return NotFound("Питання не
знайдено");

            _context.Questions.Remove(q);
            _context.SaveChanges();

            return Ok("Питання видалено");
        }
    }
}

```

Код активності LoginActivity

```

using api.Models;

public class User
{
    public int Id { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public string Password { get; set; }
}

```

```

        public int Hearts { get; set; } = 5;

        public DateTime? LastHeartUpdate { get;
        set; }

        public int Diamonds { get; set; } = 100;
        public int Streak { get; set; } = 0;
        public DateTime? LastLessonDate { get;
        set; }
    }
}

```

					КП.ОК19.ПІ.232.14.000 ПЗ	Арк.
Вим.	Арк.	№ докум.	Підпис	Дата		46

```

        public List<UserCourse> UserCourses {
get; set; }
    }

namespace api.Models
{
    public class LoginRequest
    {
        public string Email { get; set; }
        public string Password { get; set; }
    }
}

namespace api.Models
{
    public class RegisterRequest
    {
        public string Name { get; set; }
        public string Email { get; set; }
        public string Password { get; set; }
    }
}

using api.Data;
using api.Models;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;

namespace backend.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class AuthController :
ControllerBase
    {
        private readonly AppDbContext
_context;
        private readonly
PasswordHasher<User> _hasher;

        public AuthController(AppDbContext
context)
        {
            _context = context;
            _hasher = new
PasswordHasher<User>();
        }

        [HttpPost("register")]
        public IActionResult
Register([FromBody] RegisterRequest request)
        {
            if
(string.IsNullOrEmpty(request.Email) ||
string.IsNullOrEmpty(request.Password) ||
string.IsNullOrEmpty(request.Name))
            {
                return BadRequest("Заповни
всі поля");
            }

```

```

            if (_context.Users.Any(u =>
u.Email == request.Email))
                return BadRequest("Такий
email вже існує");

            var user = new User
            {
                Name = request.Name,
                Email = request.Email
            };

            user.Password =
_hasher.HashPassword(user,
request.Password);

            _context.Users.Add(user);
            _context.SaveChanges();

            return Ok(new
            {
                user.Id,
                user.Name,
                user.Email,
                user.Hearts,
                user.Diamonds,
                user.Streak
            });
        }

        [HttpPost("login")]
        public IActionResult
Login([FromBody] LoginRequest request)
        {
            if
(string.IsNullOrEmpty(request.Email) ||
string.IsNullOrEmpty(request.Password))
            {
                return BadRequest("Введи
email і пароль");
            }

            var user =
_context.Users.FirstOrDefault(u => u.Email
== request.Email);

            if (user == null)
                return
BadRequest("Користувача не знайдено");

            var result =
_hasher.VerifyHashedPassword(user,
user.Password, request.Password);

            if (result ==
PasswordVerificationResult.Failed)
                return BadRequest("Невірний
пароль");

            return Ok(new
            {
                user.Id,
                user.Name,
                user.Email,
                user.Hearts,

```

Вим.	Арк.	№ докум.	Підпис	Дата


```

        user.Diamonds,
        user.Streak
    });
}
}
}

```

Код активності ShopActivity

```

namespace api.Models
{
    public class ShopItem
    {
        public int Id { get; set; }

        public string Name { get; set; }

        public string Type { get; set; }

        public int Price { get; set; }

        public int Value { get; set; }

        public int? CourseId { get; set; }
    }
}

namespace api.DTO
{
    public class BuyDTO
    {
        public int UserId { get; set; }
        public int ItemId { get; set; }
    }
}

using api.Data;
using api.DTO;
using api.Models;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using System.Globalization;
using System.Security.Cryptography;
using System.Text;
using System.Text.Json;

namespace api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]

    public class ShopController :
        ControllerBase
    {
        private readonly AppDbContext
        _context;
        private readonly string _publicKey =
        "sandbox_i1675738884";
        private readonly string _privateKey
        =
        "sandbox_Rs6QPNmuK0sqIy3cw8FWDftULjovd17xKWB
        veOP5";

        public ShopController(AppDbContext
        context)
        {
            _context = context;
        }

        private string
        CreateSignature(string privateKey, string
        data)
        {
            var rawSignature = privateKey +
            data + privateKey;
            using (var sha1 =
            System.Security.Cryptography.SHA1.Create())
            {
                byte[] hashBytes =
                sha1.ComputeHash(Encoding.UTF8.GetBytes(rawS
                ignature));

                return
                Convert.ToBase64String(hashBytes);
            }
        }

        [HttpGet]
        public IActionResult GetAll()
        {
            var shopItems =
            _context.ShopItems
                .Select(x => new ShopItemDTO
                {
                    Id = x.Id,
                    Name = x.Name,
                    Type = x.Type,
                    Price = x.Price,
                    Value = x.Value,
                    CourseId = x.CourseId
                })
                .ToList();

            var courses = _context.Courses
                .Select(c => new ShopItemDTO
                {
                    Id = -c.Id,
                    Name = c.Name,
                    Type = "course",
                    Price = 1000,
                    Value = 0,
                    CourseId = c.Id
                })
                .ToList();

            var result =
            shopItems.Concat(courses).ToList();

            return Ok(result);
        }

        [HttpPost("buy")]
        public IActionResult Buy([FromBody]
        BuyDTO dto)
        {
            var user =
            _context.Users.FirstOrDefault(u => u.Id ==
            dto.UserId);

```

Вим.	Арк.	№ докум.	Підпис	Дата

КП.ОК19.ПІ.232.14.000 ПЗ

Арк.

48

```

        if (user == null) return
BadRequest();

        if (dto.ItemId < 0)
        {
            int courseId =
Math.Abs(dto.ItemId);

            bool owned =
_context.UserCourses
                .Any(x => x.UserId ==
dto.UserId && x.CourseId == courseId);

            if (owned)
                return Ok(new { success
= false, message = "Вже куплено" });

            if (user.Diamonds < 1000)
                return Ok(new { success
= false, message = "Недостатньо 💎" });

            user.Diamonds -= 1000;

            _context.UserCourses.Add(new
UserCourse
            {
                UserId = dto.UserId,
                CourseId = courseId
            });

            _context.SaveChanges();

            return Ok(new { success =
true, diamonds = user.Diamonds });
        }

        var item =
_context.ShopItems.FirstOrDefault(x => x.Id
== dto.ItemId);
        if (item == null)
            return Ok(new { success =
false, message = "Item не знайдено" });

        Console.WriteLine("TYPE: " +
item.Type);

        if (item.Type.ToLower() ==
"hearts")
        {
            if (user.Diamonds <
item.Price)
                return Ok(new { success
= false, message = "Недостатньо 💎" });

```

```

            user.Diamonds -= item.Price;
            user.Hearts += item.Value;
        }
        else if (item.Type.ToLower() ==
"donate")
        {
            user.Diamonds += item.Value;
        }
        else
        {
            return Ok(new { success =
false, message = "Невідомий тип: " +
item.Type });
        }

        _context.SaveChanges();

        return Ok(new
        {
            success = true,
            diamonds = user.Diamonds,
            hearts = user.Hearts
        });
    }
    [HttpPost]
    public IActionResult
Create([FromBody] ShopItemDTO dto)
    {
        if (dto == null ||
string.IsNullOrEmpty(dto.Name))
        {
            return BadRequest(new {
message = "Дані неповні" });
        }

        var newItem = new ShopItem
        {
            Name = dto.Name,
            Type = dto.Type,
            Price = dto.Price,
            Value = dto.Value,
            CourseId = dto.CourseId
        };

        _context.ShopItems.Add(newItem);
        _context.SaveChanges();

        return Ok(new { success = true,
id = newItem.Id });
    }
}

```

Код активності ProgresActivity

```

namespace api.Models
{
    public class Progress
    {
        public int Id { get; set; }

        public int UserId { get; set; }

```

```

        public int LessonId { get; set; }

        public int CorrectAnswers { get;
set; }

        public int TotalQuestions { get;
set; }

```

Вим.	Арк.	№ докум.	Підпис	Дата

КП.ОК19.ПІ.232.14.000 ПЗ

Арк.

49

```

        public int Percentage { get; set; }

        public bool IsCompleted { get; set; }
    }

    public int Reward { get; set; }
}

namespace api.DTO
{
    public class ProgressDTO
    {
        public int UserId { get; set; }
        public int LessonId { get; set; }

        public int CorrectAnswers { get;
set; }
        public int TotalQuestions { get;
set; }
    }
}
namespace api.DTO
{
    public class SaveProgressDTO
    {
        public int UserId { get; set; }
        public int LessonId { get; set; }
        public int CorrectAnswers { get;
set; }
        public int TotalQuestions { get;
set; }
    }
}
using api.Data;
using api.DTO;
using api.Models;
using Microsoft.AspNetCore.Mvc;

namespace api.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProgressController :
ControllerBase
    {
        private readonly AppDbContext
_context;

        public
ProgressController(AppDbContext context)
        {
            _context = context;
        }
        [HttpPost]
        public IActionResult
UpdateProgress([FromBody] ProgressDTO dto)
        {
            var existing =
_context.Progresses
                .FirstOrDefault(p =>
p.UserId == dto.UserId && p.LessonId ==
dto.LessonId);

```

```

            int newPercentage =
(int)((double)dto.CorrectAnswers /
dto.TotalQuestions * 100);

            int oldPercent =
existing?.Percentage ?? 0;

            if (existing == null)
            {
                existing = new Progress
                {
                    UserId = dto.UserId,
                    LessonId = dto.LessonId,
                    CorrectAnswers =
dto.CorrectAnswers,
                    TotalQuestions =
dto.TotalQuestions,
                    Percentage =
newPercentage,
                    IsCompleted =
newPercentage >= 60
                };

                _context.Progresses.Add(existing);
            }
            else
            {
                if (newPercentage >
existing.Percentage)
                {
                    existing.CorrectAnswers
= dto.CorrectAnswers;
                    existing.TotalQuestions
= dto.TotalQuestions;
                    existing.Percentage =
newPercentage;

                    if (newPercentage >= 60)
                        existing.IsCompleted
= true;
                }
            }

            var user =
_context.Users.FirstOrDefault(u => u.Id ==
dto.UserId);

            int reward = 0;

            if (user != null &&
newPercentage >= 60)
            {
                int diff = newPercentage -
oldPercent;

                if (diff > 0)
                {
                    reward = diff;
                    user.Diamonds += reward;
                }

                var today =
DateTime.UtcNow.Date;

```

Вим.	Арк.	№ докум.	Підпис	Дата

КП.ОК19.ПІ.232.14.000 ПЗ

Арк.

50

```

        if (user.LastLessonDate ==
null)
        {
            user.Streak = 1;
        }
        else
        {
            var lastDate =
user.LastLessonDate.Value.Date;

            if (lastDate == today)
            {
                // нічого
            }
            else if (lastDate ==
today.AddDays(-1))
            {
                user.Streak += 1;
            }
            else
            {
                user.Streak = 1;
            }
        }

        user.LastLessonDate = today;
    }

    _context.SaveChanges();

    return Ok(new
    {
        percentage = newPercentage,
        isCompleted =
existing.IsCompleted,
        reward = reward,
        streak = user?.Streak ?? 0
    });
}

[HttpGet("with-
progress/{courseId}/{userId}")]
public IActionResult
GetLessonsWithProgress(int courseId, int
userId)
{
    var lessons = _context.Lessons
        .Where(l => l.CourseId ==
courseId)

        .OrderBy(l => l.Id)
        .ToList();

    var lessonIds = lessons.Select(l
=> l.Id).ToList();
    var progresses =
_context.Progresses
        .Where(p => p.UserId ==
userId && lessonIds.Contains(p.LessonId))
        .ToList();

    var result = new List<object>();

    for (int i = 0; i <
lessons.Count; i++)
    {

```

```

        var currentLesson =
lessons[i];

        bool unlocked = i == 0;

        if (i > 0)
        {
            var prevLessonId =
lessons[i - 1].Id;
            unlocked =
progresses.Any(p => p.LessonId ==
prevLessonId && p.IsCompleted);
        }

        var progress =
progresses.FirstOrDefault(p => p.LessonId ==
currentLesson.Id);

        result.Add(new
        {
            id = currentLesson.Id,
            title =
currentLesson.Title,
            unlocked = unlocked,
            isCompleted =
progress?.IsCompleted ?? false,
            percentage =
progress?.Percentage ?? 0
        });
    }

    return Ok(result);
}

[HttpGet("stats/{userId}")]
public IActionResult GetStats(int
userId)
{
    var data = _context.Progresses
        .Where(p => p.UserId ==
userId)

        .Join(_context.Lessons, p =>
p.LessonId, l => l.Id, (p, l) => new { p, l
})

        .Join(_context.Courses, pl
=> pl.l.CourseId, c => c.Id, (pl, c) => new
{ c.Name, pl.p.CorrectAnswers,
pl.p.TotalQuestions })
        .ToList();

    var result = data.GroupBy(x =>
x.Name)

        .Select(g => new
        {
            course = g.Key,
            correct = g.Sum(x =>
x.CorrectAnswers),
            total = g.Sum(x =>
x.TotalQuestions)
        });

    return Ok(result);
}

[HttpGet("{id}")]
public IActionResult GetUser(int id)

```

Вим.	Арк.	№ докум.	Підпис	Дата

КП.ОК19.ПІ.232.14.000 ПЗ

Арк.

51

```

        {
            var u =
                _context.Users.FirstOrDefault(x => x.Id ==
                id);
            if (u == null) return
                NotFound();
            return Ok(u);
        }
        [HttpGet("full/{userId}")]

        public IActionResult
        GetFullProgress(int userId)

        {

            var userCourses =
                _context.UserCourses

                .Where(uc => uc.UserId ==
                userId)

                .Select(uc => uc.CourseId)

                .ToList();

            var courses = _context.Courses

                .Where(c =>
                userCourses.Contains(c.Id))

                .ToList();

            var progresses =
                _context.Progresses

                .Where(p => p.UserId ==
                userId)

                .ToList();

            var lessons =
                _context.Lessons.ToList();

            var result =
                courses.Select(course =>

                {

                    var courseLessons =
                        lessons.Where(l => l.CourseId ==
                        course.Id).ToList();

                    int lessonsTotal =
                        courseLessons.Count;

```

```

                    int lessonsDone =
                        courseLessons.Count(l =>

                            progresses.Any(p =>
                                p.LessonId == l.Id && p.IsCompleted));

                    var courseProgress =
                        progresses

                            .Where(p =>
                                courseLessons.Any(l => l.Id == p.LessonId))

                            .ToList();

                    int correct =
                        courseProgress.Sum(p => p.CorrectAnswers);

                    int total =
                        courseProgress.Sum(p => p.TotalQuestions);

                    return new

                    {

                        course = course.Name,

                        correct,

                        total,

                        lessonsDone,

                        lessonsTotal

                    };

                });

            return Ok(result);

        }

    }

    using api.Data;
    using api.Models;
    using Microsoft.AspNetCore.Mvc;

    namespace api.Controllers
    {
        [ApiController]
        [Route("api/[controller]")]

```

```

    public class HeartController :
ControllerBase
    {
        private readonly AppDbContext
_context;

        private const int MAX_HEARTS = 5;
        private const int REGEN_MINUTES = 5;

        public HeartController(AppDbContext
context)
        {
            _context = context;
        }

        [HttpGet("{userId}")]
        public IActionResult GetHearts(int
userId)
        {
            var user =
_context.Users.FirstOrDefault(x => x.Id ==
userId);
            if (user == null) return
NotFound();

            RegenerateHearts(user);

            _context.SaveChanges();

            return Ok(new
            {
                hearts = user.Hearts,
                lastUpdate =
user.LastHeartUpdate
            });
        }
        [HttpPost("lose/{userId}")]
        public IActionResult LoseHeart(int
userId)
        {
            var user =
_context.Users.FirstOrDefault(x => x.Id ==
userId);
            if (user == null) return
NotFound();

            RegenerateHearts(user);

            if (user.Hearts <= 0)
                return BadRequest("Немає
сердцець");

            user.Hearts--;

            if (user.Hearts == MAX_HEARTS -
1)
            {
                user.LastHeartUpdate =
DateTime.Now;

```

```

            }

            _context.SaveChanges();

            return Ok(new { hearts =
user.Hearts });
        }
        [HttpGet("can-start/{userId}")]
        public IActionResult CanStart(int
userId)
        {
            var user =
_context.Users.FirstOrDefault(x => x.Id ==
userId);
            if (user == null) return
NotFound();

            RegenerateHearts(user);

            _context.SaveChanges();

            return Ok(new
            {
                canStart = user.Hearts > 0,
                hearts = user.Hearts
            });
        }

        private void RegenerateHearts(User
user)
        {
            if (user.Hearts >= MAX_HEARTS)
                return;

            if (user.LastHeartUpdate ==
null)
            {
                user.LastHeartUpdate =
DateTime.Now;
                return;
            }

            var minutesPassed =
(DateTime.Now -
user.LastHeartUpdate.Value).TotalMinutes;

            int heartsToAdd =
(int)(minutesPassed / REGEN_MINUTES);

            if (heartsToAdd > 0)
            {
                user.Hearts =
Math.Min(MAX_HEARTS, user.Hearts +
heartsToAdd);
                user.LastHeartUpdate =
DateTime.Now;
            }
        }
    }

```

